UTS

Computing Science Studio 2

Project Report

Data Structures Enabling Quantum Encrypted Multi-Cloud Storage

Aaron Abraham Thomas

Callum Burke

Jarrod Wallace

Joshua Tan

Michael Minaca

Sibishan Ravindran

School of Computer Science,

Faculty of Engineering and Information Technology,

University of Technology Sydney

15 May 2026

UTS

Table of Contents

ABSTRACT **3**

1. INTRODUCTION **3**

2. RELATED WORK **4**

2.1 Optimal Fidelity Bound for Universal Quantum Cloning 4
2.2 Prior Cloning Methodologies 5
2.3 Perfect and Deterministic Cloning Method 6
2.3.1 Encrypted Cloning Method 6
2.3.2 Decrypting the Qubit 7
2.4 Research Gap 8

3. PROBLEM DEFINITION **9**

3.1 Clear Problem Formula 9
3.2 Dataset Collection and Analysis 10

4. FRAMEWORK **10**

4.1 Framework Design 10
4.1.1 Protocol Design 11
4.1.2 QStack Design 12
4.1.3 QArray Design 13
4.2 Framework Implementation 15
4.2.1 Protocol Implementation 15
4.2.2 QStack Implementation 24
4.2.3 QArray Implementation 29

5. EXPERIMENTAL ANALYSIS **41**

5.1 Experimental Environment 41
5.2 Evaluation Criteria 42
5.3 Validation Study 43
5.4 Performance Analysis 46
5.5 Comparison against the Theoretical and Experimental Baselines 48

6. DISCUSSION **49**

7. CONCLUSION **51**

REFERENCES **53**

Abstract

The no-cloning theorem prevents arbitrary quantum states from being copied. This poses a fundamental obstacle to the implementation of redundant quantum cloud storage. Yamaguchi and Kempf (2026) recently introduced an encrypted cloning protocol that bypasses this constraint by producing n encrypted clones of an unknown state, and one of them can be deterministically decrypted with fidelity 1.0. However, no software framework exists for managing the lifecycle of such clones at the application level, leaving end users to manually construct the protocol's gate-level circuits. This study addresses that gap by designing and implementing two data structure abstractions. One is QArray, which provides indexed access, and the other is QStack, which provides last-in-first-out (LIFO) access. Both data structures are built on top of a Protocol class that encapsulates the Qiskit quantum operations of the encrypted cloning protocol. The framework enforces the protocol's one-time decryption constraint, along with the no-cloning and no-measurement constraints. We validate the framework across 800 ideal simulation trails, 400 noisy simulation trails under an IBM Heron R2 depolarising noise model, and resource scaling sweeps up to 28 qubits. Under ideal simulations, retrieval fidelity reaches 1.0, individual qubit purity matches the theoretical 0.5, and total qubit count adheres to the $m(2n + 1)$ bound. On the other hand, noisy simulation fidelity reproduces the degradation trend observed on real quantum hardware. These results demonstrate that high-level data-structure abstractions for encrypted quantum storage are viable on current simulators, providing a reusable software layer for future research on quantum encrypted multi-cloud storage.

1. Introduction

In quantum mechanics, the no-cloning theorem states that under no circumstances can an arbitrary unknown quantum state be perfectly cloned. Therefore, quantum computers cannot copy an arbitrary qubit into another qubit (Nielsen & Chuang, 2010). Consequently, quantum cloud storage can play a vital role in storing quantum states, as it is impossible to store unknown arbitrary quantum states classically. This is because once we measure a qubit, it loses its quantum properties and results in either 1 or 0. It is impossible to reconstruct an arbitrary quantum state from the measurement (Nielsen &

Chuang, 2010). Yamaguchi and Kempf (2026) have shown that an encrypted qubit can be cloned n times, but only one can be decrypted while obeying the no-cloning theorem. They proposed quantum encrypted multi-cloud storage as a primary application of their work.

This study aims to design and implement array and stack data structures that support the encrypted cloning protocol while addressing the primary research question: "**How can data structures for the encrypted cloning protocol efficiently enable quantum encrypted multi-cloud storage**?" Such storage could allow the owner of the quantum states to store them off-site, redundantly across multiple quantum clouds to protect against hardware failures, and to encrypt them with a key kept on-site by the owner. This has significant implications for industries that require long-term, secure storage of sensitive quantum data, including finance and national security, while raising ethical considerations around the responsible usage of this technology. A software framework for managing the lifecycle of these encrypted clones at scale is absent from the existing literature. Such a framework could allow end users to store and retrieve qubits using abstract methods without knowing the protocol at the gate level.

Our contribution is the creation of data structure abstractions for encrypted cloning protocol using arrays and stacks. The expected outcomes are:

i. A working Python implementation of array and stack data structures for the protocol.
ii. Verification that they correctly enforce the protocol's constraints such as one-time decryption.
iii. Benchmarking against the protocol's theoretical fidelity, and through evaluation of quantum resource scaling.

2. Related Work

2.1 Optimal Fidelity Bound for Universal Quantum Cloning

The fundamental constraint of quantum information theory forbids "quantum copiers" that can take one unknown quantum system and reproduce two identical outputs with the same statistical properties (Nielsen & Chuang, 2010). While the no-cloning theorem

prevents making perfect copies, quantum mechanics allows for optimal approximate clones. According to Bužek and Hillery (1996), and Werner (1998), the shrinking factor γ, which quantifies how faithfully the cloner preserves the original state, is given by

$$\gamma_{opt}(\mathrm{N},\mathrm{M},\mathrm{d}) = \frac{N}{d+N}\frac{d+M}{M}, \qquad (1)$$

where N is the number of input copies of the unknown state $|\psi\rangle \in \mathbb{C}^d$, $M \geq N$ is the number of approximate output clones, and d is the dimension of the Hilbert space of each system. For a $1 \rightarrow 2$ qubit cloner where $N = 1,\ M = 2,\ d = 2$, this gives $\gamma_{opt} = \frac{2}{3}$ (Bužek & Hillery, 1996). The individual fidelity is then derived from the formula

$$R\left(T\left(\sigma^{\otimes N}\right)\right) = \gamma\sigma + (1-\gamma)\tau_1, \qquad (2)$$

where $\sigma = |\psi\rangle\langle\psi|$ is the pure input state, T denotes the cloning channel acting on N input copies, R is the partial trace over the remaining $M-1$ clones, and $\tau_1 = \frac{I}{d}$ is the maximally mixed state on a single system. Taking the overlap with $|\psi\rangle$ gives

$$F_{single} = \gamma + \frac{1-\gamma}{d}, \qquad (3)$$

substituting in the values gives $\frac{2}{3} + \frac{1-\frac{2}{3}}{2} = \frac{2}{3} + \frac{1}{6} = \frac{5}{6}$ (Werner, 1998). This establishes that the optimal fidelity of any universal $1 \rightarrow 2$ qubit cloning method cannot surpass $\frac{5}{6}$.

2.2 Prior Cloning Methodologies

Other cloning methodologies exist, such as broadcasting of mixed-state cloning, imperfect cloning, and probabilistic cloning. According to Barnum et al. (1996), broadcasting is the attempt to distribute the information about a quantum state to multiple systems while preserving the state locally on each system. The basic idea behind this method is as follows: Suppose there exists a quantum system A that is in an unknown mixed state described by a density matrix ρ. We want to create another system B such that both A and B appear in the state ρ.

Imperfect cloning addresses the concept of quantum copying and the fundamental limits imposed by quantum mechanics. Bužek and Hillery (1996), discussed in Section 3.1, state that the ideal copying process would take the original quantum state and produce an identical copy of it, outputting both the original and the copy. However, the no-cloning

theorem prohibits perfect copying of an unknown state instead produces approximate clones with bounded fidelity loss.

Duan and Guo (1998) construct a probabilistic quantum cloning machine utilising a general unitary-reduction operation. Although the no-cloning theorem forbids the copying of unknown quantum states, perfect cloning can still occur probabilistically. Duan and Guo (1998) prove that a set of states can still be cloned with non-zero probability only if the states are linearly independent, and they determine the optimal success probabilities.

Broadcasting and imperfect cloning are fundamentally limited in their ability to produce faithful copies of arbitrary quantum states, while probabilistic cloning achieves perfect fidelity only with a non-guaranteed probability of success. In contrast, Yamaguchi and Kempf's (2026) encrypted cloning deterministically achieves a fidelity of 1.0, offering an opportunity to host redundant quantum-state storage.

2.3 Perfect and Deterministic Cloning Method

Yamaguchi and Kempf (2026) have recently demonstrated that, although the no-cloning theorem blocks direct copying of unknown quantum states, qubits can be cloned using encrypted cloning method. The broad process is laid out below:

2.3.1 Encrypted Cloning Method

First, they start by maximally entangling two groups of n qubits, signal qubit (S) and noise qubit (N), such that

$$\forall i \in \{1, \dots, n\}, \qquad |\phi\rangle_{S_i N_i} = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle). \tag{4}$$

They define the encrypted cloning operation below, where $\sigma_1 = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\sigma_3 = Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, and A is the qubit that is being cloned:

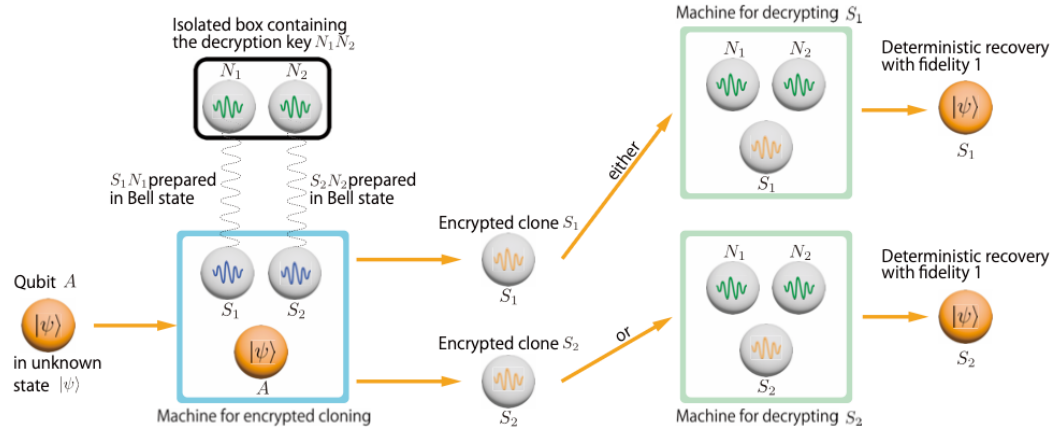
$$U_{enc}^{(n)} := e^{-\frac{\pi i}{4}\sigma_1^{(A)} \otimes \left(\otimes_{i=1}^{n} \sigma_1^{(S_i)}\right)} e^{-\frac{\pi i}{4}\sigma_3^{(A)} \otimes \left(\otimes_{i=1}^{n} \sigma_3^{(S_i)}\right)}, \tag{5}$$

This cloning operator $U_{enc}^{(n)}$ is then applied to A and S. When the cloning operator is applied, A interacts with the signal qubits $\{S_1, S_2, \dots, S_n\}$. This causes the state of A to be

imprinted into the signal qubits, leaving us with n encrypted clones, while qubit A becomes maximally mixed and can be discarded. If any single signal qubit was to be observed, it would be a mixed state. Thus, encrypted. The cloning operator leaves the noise qubits $\{N_1, N_2, \dots, N_n\}$ untouched, which will be used as the decryption key in the following step.

Figure 1

Encrypted cloning of an unknown quantum state procedure.



Source: (Yamaguchi & Kempf, 2026)

2.3.2 Decrypting the Qubit

They have illustrated in figure 1 that only one of the n encrypted clones can be decrypted to obtain the original quantum state of A. This is the primary constraint of this cloning protocol by design. The decryption achieves fidelity of 1, perfectly recovering the original quantum state. This remains consistent with the no-cloning theorem because only one encrypted clone can be decrypted. They define decrypting a clone operation below, where $\alpha_0 = 1, \alpha_1 = \alpha_3 = i, \alpha_3 = -i^{n+1}, \sigma_0 = I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \sigma_2 = Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, and S_1 is selected to be decrypted:

$$U_{dec}^{(n)} := \sum_{\mu=0}^{3} \alpha_\mu \left(|\phi_\mu\rangle\langle\phi_\mu|_{S_1N_1}\right) \otimes \left(\bigotimes_{j=2}^{n} \sigma_\mu^{(N_j)^\top} \right), \qquad (6)$$

This decrypting operator requires all the noise qubits to be consumed as decryption key to decrypt the selected clone. After obtaining the original state back, the rest of the signal qubits, in this case $\{S_2, S_3, \dots, S_n\}$, no longer contain the original quantum state imprinted

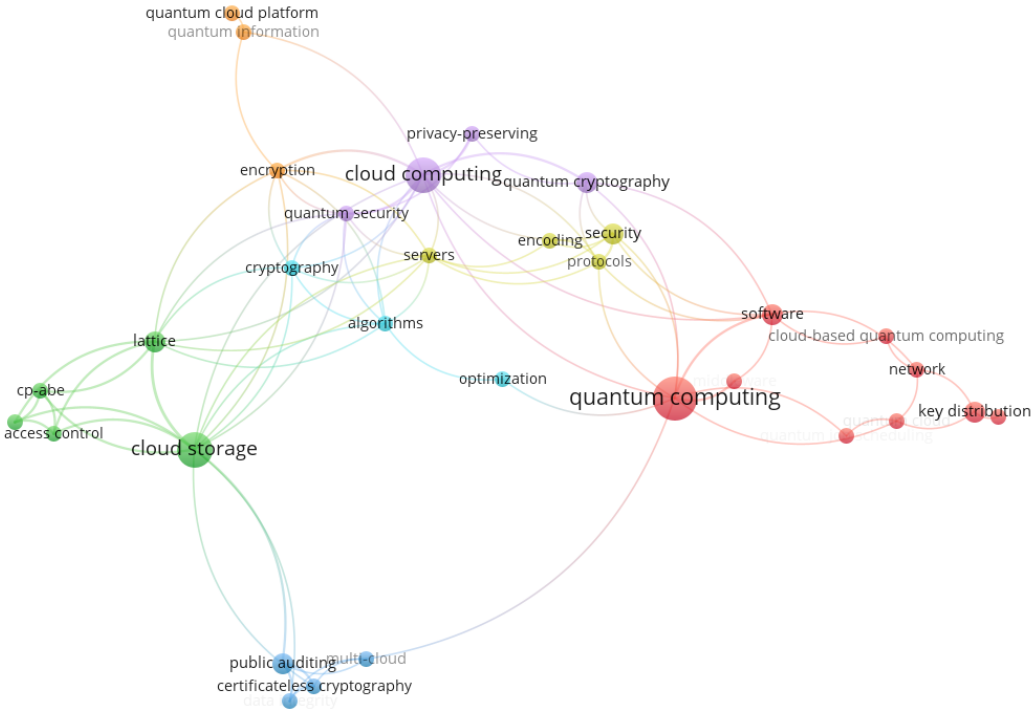
in them. Therefore, we can dispose the noise and signal qubits after decryption, leaving only the decrypted qubit in the system.

2.4 Research Gap

There appears to be a gap between quantum computing and cloud storage, primarily due to the no-cloning theorem. Figure 2 shows that there is no direct connection between quantum computing and cloud storage in the literature dataset obtained from the Web of Science Core Collection database. Yamaguchi and Kempf's (2026) work on cloning encrypted qubits provides a protocol that achieves deterministic cloning fidelity of 1, unlike other cloning methodologies. Fujiwara et al. (2016) and Ma et al. (2023) demonstrated that we can securely store and retrieve classical data on a classical storage server using quantum encryption protocols. While QRAM addresses short-term quantum memory for algorithm execution (Phalak et al., 2023), long-term distributed quantum storage remains unexplored. Yamaguchi and Kempf (2026) proposed quantum-encrypted multi-cloud storage as an application of their protocol but did not provide an implementation. Our study aims to fill this gap by developing data structures that provide an application-level schema for quantum encrypted multi-cloud storage.

Figure 2

Co-occurrence of keywords analysis on literature dataset.



3. Problem Definition

3.1 Clear Problem Formula

We formulate the problem of designing the abstraction of Yamaguchi and Kempf's (2026) encrypted cloning protocol using data structures as follows. Let m denote the storage capacity and n the redundancy level. A data structure $D = \langle A, S, N, L \rangle$ consists of m data qubit registers $A = \{A, \dots, A_m\}$, an $m \times n$ matrix S of signal qubit registers where $S[i, j]$ is the j^{th} encrypted clone of slot i, an $m \times n$ matrix N of corresponding noise qubit registers serving as decryption keys, and a classical lookup table L tracking slot occupancy. We define two such data structures. One is QArray and the other is QStack. QArray supports indexed access through set$(i, |\psi\rangle)$, get(i, c), insert$(i, |\psi\rangle)$, append$(|\psi\rangle)$, remove(i), and reverse$()$, where $i \in \{0, \dots, m-1\}$ indexes a slot and $c \in \{0, \dots, n-1\}$ selects a clone to decrypt. On the other hand, QStack supports last-in-first-out access through push$(|\psi\rangle)$

and pop(c). Each operation reduces to at least one of the Protocol's calls, such as store_qubit, retrieve_qubit, swap_a, and uncompute_a.

The framework must enforce three invariants derived from the protocol and from quantum mechanics. First, the protocol's one-time decryption constraint, where for any slot i at most one of the n signal-noise pairs $(S[i,j], N[i,j])$ shall be consumed by retrieval. Second, the no-cloning constraint, which forces mutation operations such as insert, remove, and reverse to be implemented as retrieve-swap-store cycles rather than copies. Third, the no-measurement constraint, which rules out operations such as peek or contains, since mid-computation measurement collapses the superposition and renders the protocol non-functional thereafter (Nielsen & Chuang, 2010). The scope of this work is bounded to the implementation of the Protocol and the two data structures (QArray and QStack).

3.2 Dataset Collection and Analysis

As the encrypted cloning protocol lacks a quantum circuit model implementation, there is no pre-existing dataset to analyse prior to design and development. Therefore, we need to generate evaluation data by simulating our framework across multiple configurations. The simulations will include all components of our framework: the protocol, array, and stack. The collection and analysis of these results will be discussed in depth in Section 6 (Experimental Analysis), covering the experimental environment, evaluation criteria, validation study across configurations, and performance benchmarking. The following section covers our framework design and implementation before moving to the experimental analysis.

4. Framework

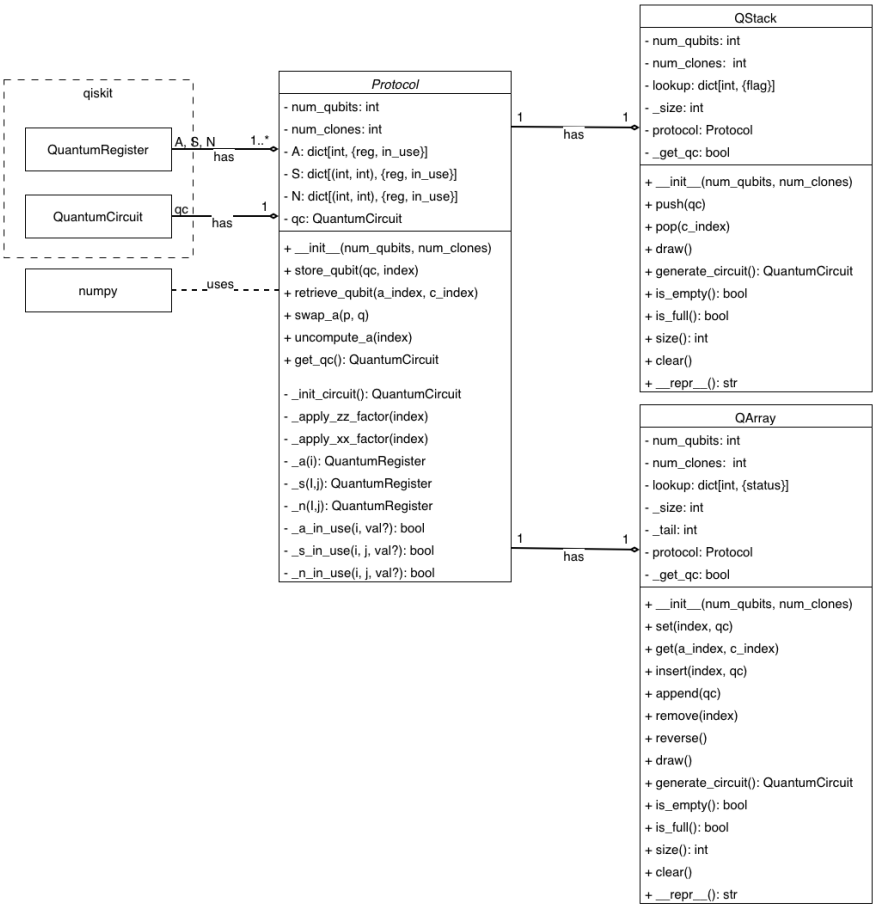
4.1 Framework Design

Figure 3 illustrates our overall framework. The framework consists of three core components, which are the Protocol, QStack, and QArray classes. The protocol class implements the encrypted cloning protocol's encryption and decryption operations. The QStack class offers last-in-first-out access and QArray class provides indexed storage and retrieval of multiple quantum states. Each data structure delegates its underlying

quantum operations to the Protocol class, allowing users to store and retrieve quantum states without interacting with the protocol directly. The following sections discuss the design decisions for each class.

Figure 3

UML class diagram of the framework.



4.1.1 Protocol Design

The Protocol class serves as a software encapsulation for Yamaguchi and Kempf's (2026) encrypting operation defined in Equation 5, which stores any given quantum state in n signal qubits (S) thus creating n clones of that state, and decrypting operation defined in Equation 6, which retrieves the stored quantum state from only one of the signal qubits (S) and consumes all its counterpart noise qubits (N) as one time decryption key. To store all these quantum operations, we use the quantum circuit, also known as the quantum

program, from Qiskit to run it on quantum computers or simulators after compilation (Nielsen & Chuang, 2010). To make the protocol store multiple quantum states with n clones in a circuit, we need two parameters: the number of qubits and the number of clones. When a protocol object is constructed, we also create 3 mappings to store quantum registers for qubit (A), signal qubit (S), and noise qubit (N). Along with storing quantum registers, the mapping also stores a flag for each register indicating whether it is in use. This bookkeeping can ensure that a one-time decryption constraint is not violated. Moreover, using a map-like data structure to store the quantum registers decouples the protocol's internal state from any specific data structure. This means additional data structures beyond QStack and QArray can be built on top of the Protocol class in the future without modifying its underlying implementation.

The public methods of the protocol can be grouped into storage (*store_qubit*), retrieval (*retrieve_qubit*), register manipulation (*swap_a, uncompute_a*), and circuit compilation (*get_qc*). On the other hand, the private methods *_apply_zz_factor* and *_apply_xx_factor* implement the two factors of the encryption unitary as decomposed by Yamaguchi and Kempf (2026) in Equation 5, rather than applying it as a single monolithic gate. This allows each factor to be directly translated into standard one- and two-qubit gates available on current quantum hardware. The private method *_init_circuit* is called once during construction to create a quantum circuit with quantum registers in the mapping of A, S, and N and implement entanglements between each signal qubit and noise qubit as defined in Equation 1. There are also additional helpers, such as *_a, _s, _n*, and their corresponding *_in_use* checks, which provide controlled access to the registers and are used to enforce constraints of the protocol.

4.1.2 QStack Design

The QStack class provides last-in-first-out (LIFO) access to quantum-encrypted storage, suitable for workloads that require the most recently stored quantum state to be accessed first. It delegates all quantum operations to an internal Protocol instance, meaning QStack's own overhead is purely classical bookkeeping on top of the Protocol's existing quantum resources. It is constructed with two parameters, which are *num_qubits* and *num_clones*. The *num_qubits* parameter defines the stack's capacity,

and *num_clones* sets the redundancy level for each stored state. The core operations in QStack are *push* and *pop*, and it also encompasses utility functions such as *draw*, *generate_circuit, is_empty*, *is_full*, *size*, and *clear*.

In a normal stack, when a *pop* is applied, the top element is returned to the user, removed from the stack, and a space is freed up. Unlike a normal stack, *pop* decrypts the quantum state from the selected signal qubit and returns it to the corresponding qubit A. This operation uncomputes the associated signal and noise qubits, so the slot cannot be reused for a new *push*. Therefore, the logical size of QStack can never be reduced. A lookup mapping with a flag for each slot tracks which positions are occupied, enabling the protocol to route each retrieval to the correct qubit A. Meanwhile, the *_size* serves both as the element count and the stack pointer for the next *push*. Moreover, the *pop* also requires a *c_index* parameter because the protocol's one-time decryption constraint requires the user to specify which of the n encrypted clones to decrypt. On the other hand, the *push* in QStack behaves similarly to a normal stack, where the user must pass a single qubit quantum circuit containing the quantum state. Furthermore, standard overflow and underflow protections are enforced on *push* and *pop*, respectively. Once *generate_circuit* is called to extract the underlying quantum circuit, a Boolean lock prevents further mutations, ensuring the circuit's integrity is preserved for compilation and execution.

Furthermore, QStack does not implement functions such as *peek* or *contains* because they require reading the data, and in quantum computation, quantum registers can only be measured at the end of the computation. Taking a measurement in the middle of the computation will collapse the quantum state, which destroys the superposition, and the protocol will not be functional thereafter (Nielsen & Chuang, 2010). Therefore, such measurement-requiring functions should not be implemented.

4.1.3 QArray Design

The QArray class provides indexed access to quantum-encrypted storage, suitable for workloads that require retrieving arbitrary quantum states by position. Similar to QStack, it delegates all quantum operations to an internal Protocol instance and is constructed with two parameters: the number of qubits and the number of clones per qubit. The

number of qubits parameter defines the array's capacity, and the number of clones parameter sets the redundancy level per qubit storage. The core operations in QArray are *set* and *get*, and it also encompasses mutation methods such as *insert*, *append*, *remove*, *reverse*, as well as utility functions such as *draw*, *generate_circuit*, *is_empty*, *is_full*, *size*, and *clear*.

In a normal array, elements can be freely read, overwritten, and rearranged. Unlike a normal array, QArray's mutation operations, such as insertion, removal, and reversal, require retrieve-swap-store cycles through the protocol, because quantum states cannot be copied. This makes them more computationally intensive than their classical counterparts, a necessary trade-off for the flexibility of random access. Since QArray is a random-access storage, the *set* and *get* methods require a slot index as an argument to store and retrieve quantum states. Moreover, similar to QStack's *pop*, QArray's get also requires an index argument to retrieve the quantum state from a specific clone. Furthermore, as with QStack, the get method returns the quantum state to the corresponding qubit A register; therefore, the array size never decreases. However, QArray also uses the *remove* function to clear a slot of the user's choice. When a slot is removed, the array has to shift the following elements to the left by one if we were visualising the QArray in a horizontal plane. To shift slots in QArray, it employs a retrieve-swap-store cycle, as required by the no-cloning theorem. The QArray also needs to consider if the *get* method has already been used on the slot during the retrieve-swap-store cycle, as we do not want to store a qubit that has already been retrieved. Therefore, we also employed a lookup table to store the status of each slot and indicate whether each slot contains a qubit. This lookup table comes in handy for functions that require retrieve-swap-store cycles, such as *remove*, *insert*, and *reverse*. The *_tail* attribute tracks the highest occupied index plus one, serving as the upper bound for shifting operations and the insertion point for append. Meanwhile, *_size* tracks the number of currently stored elements independently of *_tail*, since the user can set elements at arbitrary indices, potentially leaving intermediate slots empty.

Furthermore, like QStack, QArray also provides standard overflow and underflow protections that are enforced on *set* and *get*, respectively. Once *generate_circuit* is called to extract the underlying quantum circuit, a Boolean lock prevents further mutations,

ensuring the circuit's integrity is preserved for compilation and execution. As with QStack, QArray does not implement measurement-requiring functions such as contains or index search, since mid-computation measurement would collapse the quantum state, destroying the superposition, and the protocol would not be functional thereafter (Nielsen & Chuang, 2010).

4.2 Framework Implementation

Our framework is built on Python, chosen for its extensive quantum computing ecosystem and accessibility to researchers. The protocol class is built on Qiskit, an open-source Python-based software development kit (SDK) developed by IBM for working with quantum computers at the register level. We also use NumPy to construct the decryption unitary defined in Equation 6, as Qiskit's unitary gate interface accepts NumPy arrays directly. Our data structures, QStack and QArray, are built on the Protocol, delegating all quantum operations to it as described in the design sections. The following subsections present our implementations in pseudocode, and the full codebase can be found at https://github.com/sibishan/encrypted-cloning-data-structures.

4.2.1 Protocol Implementation

4.2.1.1 Protocol Initialisation

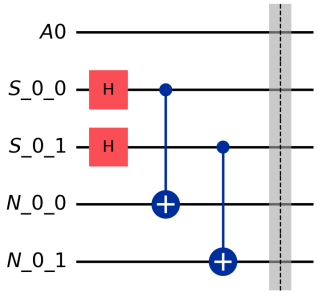
The constructor defined in Algorithm 1 initialises the Protocol by allocating quantum registers and constructing the base quantum circuit. For each of the *num_qubits* slots, one data qubit register A is created, along with *num_clones* pairs of signal (S) and noise (N) registers, all stored in mappings with their *in_use* flags set to false. These flags allow the methods in the protocol to validate constraints such as the one-time decryption of stored quantum states. The constructor then delegates to *_init_circuit* to collect all registers into a single QuantumCircuit and entangle each (S, N) pair using a Hadamard gate followed by a CNOT, as required by Equation 4. A barrier is placed after the Bell-pair preparation layer to visually separate it from subsequent operations. Figure 4 illustrates the quantum circuit produced when the protocol is called upon with the number of qubits equal to 1 and the number of clones per qubit equal to 2.

Algorithm 1 Protocol Initialisation

```
Class Protocol
    procedure __INIT__(n_qubits, n_clones)
        num_qubits ← n_qubits
        num_clones ← n_clones
        A ← {}
        for i = 0 to num_qubits − 1 do
            A[i] ← {reg : QuantumRegister(1, A_i), in_use : ⊥}
        end for
        S ← {},   N ← {}
        for i = 0 to num_qubits − 1 do
            for j = 0 to num_clones − 1 do
                S[i, j] ← {reg : QuantumRegister(1, S_{i,j}), in_use : ⊥}
                N[i, j] ← {reg : QuantumRegister(1, N_{i,j}), in_use : ⊥}
            end for
        end for
        qc ← INITCIRCUIT
    end procedure

    function INITCIRCUIT
        R ← {A[i].reg}_i ∪ {S[i, j].reg}_{i,j} ∪ {N[i, j].reg}_{i,j}
        qc ← QuantumCircuit(R)
        for i = 0 to num_qubits − 1 do
            for j = 0 to num_clones − 1 do
                qc.H(S[i, j])
                qc.CNOT(S[i, j], N[i, j])
            end for
        end for
        qc.barrier()
        return qc
    end function
EndClass
```

Figure 4

Protocol initialised with num_qubits=1 and num_clones=2.



4.2.1.2 Protocol Utilities

The utility methods defined in Algorithm 2 provide a clean internal interface to the lookup tables, also known as mapping, which consists of quantum registers and *in_use* flags. We have three mappings for A, S, and N, rather than accessing them directly throughout the protocol code. All higher-level methods, such as *store_qubit*, *retrieve_qubit*, and *swap_a*, use helper functions such as *_a*, *_s*, *_n*, *_a_in_use*, *_s_in_use*, and *_n_in_use*. Qiskit's gate methods require a qubit object rather than the register itself. Therefore, our accessor functions *_a*, *_s*, *_n* each return the underlying qubit object from the register. The accessor functions *_a_in_use*, *_s_in_use*, and *_n_in_use* serve as both getters and setters. When a Boolean value is passed as an argument to one of the *_in_use* methods, it behaves as a setter, otherwise, it is a getter. This simplifies our codebase and makes it easier to debug the protocol. Finally, *get_qc* returns the quantum circuit for compilation and execution. These utilities contain no validations, as they are internal helpers and can be trusted to be called with valid indices by higher-level methods that have already performed bounds checking.

Algorithm 2 Protocol Utilities

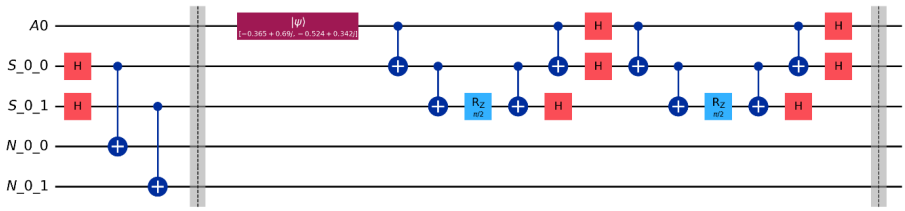
```
Class Protocol
    function a(i)
        return A[i].reg[0]
    end function

    function s(i, j)
        return S[i, j].reg[0]
    end function

    function n(i, j)
        return N[i, j].reg[0]
    end function

    function a_in_use(i, val = ∅)
        if val ≠ ∅ then
            A[i].in_use ← val
        end if
        return A[i].in_use
    end function

    function s_in_use(i, j, val = ∅)
        if val ≠ ∅ then
            S[i, j].in_use ← val
        end if
        return S[i, j].in_use
    end function

    function n_in_use(i, j, val = ∅)
        if val ≠ ∅ then
            N[i, j].in_use ← val
        end if
        return N[i, j].in_use
    end function

    function GET_QC
        return qc
    end function
EndClass
```

4.2.1.3 Storing a Quantum State

The Algorithm 3 implements the *store_qubit* method for the encrypted cloning of a given quantum state in qubit A to its corresponding signal qubits S as defined by Yamaguchi and Kempf (2026) in Equation 5. If the caller provides a single-qubit quantum circuit, it is first composed onto the corresponding qubit A, loading the quantum state before any encrypted cloning operations are applied. However, it is not necessary to always pass the single-qubit quantum circuit because methods such as insert and reverse from QArray will use the quantum state already in qubits A. Therefore, the data structure needs to check whether a single-qubit quantum circuit is null in its methods. The encryption unitary is decomposed into two factors, implemented by *_apply_zz_factor* and *_apply_xx_factor*, respectively. The ZZ factor builds a forward CNOT ladder from qubit A through the signal chain, fanning the parity into the last signal qubit. Then applies an RZ(π/2) rotation in the last signal qubit and uncomputes the ladder in reverse. The XX factor conjugates the entire ZZ interaction into the X basis by sandwiching it between Hadamard gates on qubit A and all signal qubits. This allows us to reuse our code and improve readability in our codebase. After both factors are applied, a barrier is used to visually separate the encrypting cloning operations block from subsequent operations. Finally, the *in_use* flags for all signal qubits and noise qubits associated with qubit A are set to true, along with qubit A, which is used for storing the quantum state. This method also enforces two preconditions relating to index error to prevent invalid access, such as passing out-of-bounds index or trying to store another quantum state when there's already a quantum state stored or placed in qubit A. Figure 5 illustrates the quantum circuit after storing an arbitrary quantum state with *num_qubits*=1 and *num_clones*=2.

Figure 5

Quantum circuit after storing one quantum state into slot 0 with num_qubits=1 and num_clones=2, showing the ZZ and XX factor gate sequences applied across qubit A and signal qubits S.



UTS

Algorithm 3 Protocol Qubit Storage

```
Class Protocol
    procedure STORE_QUBIT(qc, index)
        if index < 0 or index ≥ num_qubits then
            raise IndexError
        end if
        flag ← ⊥
        for i = 0 to num_clones − 1 do
            if S_IN_USE(index, i) or N_IN_USE(index, i) then
                flag ← ⊤
                break
            end if
        end for
        if flag and A_IN_USE(index) then
            raise IndexError
        end if
        if qc ≠ ∅ then
            qc ← qc.compose(qc, A[index])
        end if
        A_IN_USE(index, ⊤)
        APPLY_ZZ_FACTOR(index)
        APPLY_XX_FACTOR(index)
        qc.barrier()
        for i = 0 to num_clones − 1 do
            S_IN_USE(index, i, ⊤)
            N_IN_USE(index, i, ⊤)
        end for
    end procedure

    procedure APPLY_ZZ_FACTOR(index)
        qc.CNOT(A[index], S[index, 0])
        for i = 0 to num_clones − 2 do
            qc.CNOT(S[index, i], S[index, i + 1])
        end for
        qc.R_z(π2, S[index, num_clones − 1])
        for i = num_clones − 1 downto 1 do
            qc.CNOT(S[index, i − 1], S[index, i])
        end for
        qc.CNOT(A[index], S[index, 0])
    end procedure

    procedure APPLY_XX_FACTOR(index)
        qc.H(A[index])
        for i = 0 to num_clones − 1 do
            qc.H(S[index, i])
        end for
        APPLY_ZZ_FACTOR(index)
        qc.H(A[index])
        for i = 0 to num_clones − 1 do
            qc.H(S[index, i])
        end for
    end procedure
EndClass
```

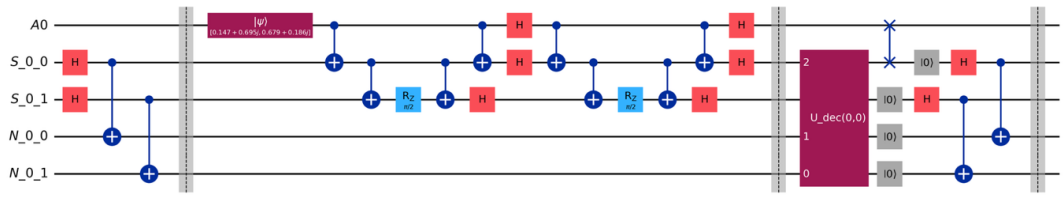
4.2.1.4 Retrieving a Stored Quantum State

Algorithm 4 depicts the *retrieve_qubit* method, which implements the remaining half of Yamaguchi and Kempf's (2026) protocol by decrypting one of the signal qubits to obtain the stored qubit. The decryption unitary is constructed numerically as the sum over μ of $\alpha\mu\left(P_\mu \otimes \Omega\mu\right)$, where P_μ is the projector onto the μ^{th} entanglement acting on the chosen clone pair ($S[a\_index, c\_index]$, $N[a\_index, c\_index]$) and $\Omega\mu$ is the tensor product of $\sigma\mu^{\intercal}$ over all remaining noise qubits $j \neq$ *c_index*, corresponding exactly to Equation 6. Qiskit unitary gate interprets qubits in least-significant-bit-first order. However, the decryption unitary matrix is constructed with the most-significant-bit-first order. Therefore, we have to pass the reversed qubit list to Qiskit's unitary gate. After the unitary is applied, a SWAP gate moves the recovered state from the selected signal qubit back into qubit A. This restores the convention that qubit A always holds the unencrypted quantum state. Finally, the signal and noise qubits are cleared, and the *in_use* flags of associated quantum registers except qubit A's are set to false and new entanglement between associated signal and noise qubits is formed according to Equation 4. We also add the barrier at the end to visually separate retrieval from any subsequent operations.

This method enforces two preconditions. First, it ensures that the given index for retrieval has its qubit A, signal qubits S, and noise qubits N in use. If the associated S and N are not in use, it means they have already been decrypted. Therefore, the protocol's one-time decryption constraint is obeyed. Second, it ensures the given index is within the mapping's bounds. Figure 6 illustrates the quantum circuit after storing and then retrieving an arbitrary quantum state from clone 0 with *num_qubits*=1 and *num_clones*=2.

Figure 6

Quantum circuit after storing one quantum state then retrieving it from clone 0, with num_qubits=1 and num_clones=2.



UTS

Algorithm 4 Protocol Qubit Retrieval

ClassProtocol
  procedure RETRIEVE_QUBIT(a_index, c_index)
    $n \leftarrow num\_clones$
    if not A_IN_USE(a_index) **then**
      raise $ValueError$
    end if
    if $c\_index < 0$ **or** $c\_index \geq n$ **then**
      raise $IndexError$
    end if
    if not S_IN_USE(a_index, c_index) **then**
      raise $ValueError$
    end if
    if not N_IN_USE(a_index, c_index) **then**
      raise $ValueError$
    end if

    $\sigma \leftarrow \{\sigma_0 = I,\ \sigma_1 = X,\ \sigma_2 = Y,\ \sigma_3 = Z\}$
    $\alpha \leftarrow \{1,\ i,\ -i^{n+1},\ i\}$
    $|\phi_0\rangle \leftarrow 1\sqrt{2}(|00\rangle + |11\rangle)$
    $|\phi_\mu\rangle \leftarrow (\sigma_\mu \otimes I)|\phi_0\rangle \quad \forall\, \mu \in \{0, 1, 2, 3\}$

    $U_{dec} \leftarrow \mathbf{0}_{2^{n+1} \times 2^{n+1}}$
    for $\mu = 0$ **to** 3 **do**
      $P_\mu \leftarrow |\phi_\mu\rangle\langle\phi_\mu|$
      $\Omega_\mu \leftarrow \bigotimes_{j \neq c\_index} \sigma_\mu^\top$
      $U_{dec} \leftarrow U_{dec} + \alpha_\mu\,(P_\mu \otimes \Omega_\mu)$
    end for

    $other_N \leftarrow \{N[a\_index,\ j] \mid j \in \{0, \ldots, n-1\},\ j \neq c\_index\}$
    $qubits \leftarrow [S[a\_index,\ c\_index],\ N[a\_index,\ c\_index]] \parallel other_N$
    $qc.unitary(U_{dec},\ qubits)$
    $qc.SWAP(A[a\_index],\ S[a\_index,\ c\_index])$

    for $i = 0$ **to** $n - 1$ **do**
      $qc.reset(N[a\_index,\ i])$
      $qc.reset(S[a\_index,\ i])$
      S_IN_USE(a_index, i, $\perp$)
      N_IN_USE(a_index, i, $\perp$)
    end for

    for $i = 0$ **to** $num\_clones - 1$ **do**
      $qc.H(S[a\_index,\ i])$
      $qc.CNOT(S[a\_index,\ i],\ N[a\_index,\ i])$
    end for
    $qc.barrier()$
  end procedure
EndClass

4.2.1.5 Managing Qubits in the Protocol

Since QArray is a random-access storage, it requires qubit management such as swapping quantum states between adjacent quantum registers in the set A and an uncomputing method to discard a stored qubit entirely by resetting qubit A along with its signal and noise qubit if it holds clones. Algorithm 5 depicts how we implemented the *swap_a* and *uncompute_a* methods. For the *swap_a* method, we assume that the data structure, such as QArray, will only use it when there are no clones present for the two-qubit arguments. We apply a SWAP operation between qubit A and also swap the *in_use* flags for valid bookkeeping. We only validate the indices of both qubits to ensure that they are within the bounds of the qubit A mappings. Figure 7 illustrates the quantum circuit after swapping 2 data qubits, which have not yet been cloned.

The *uncompute_a* method resets all qubits to $|0\rangle$ associated with the data qubit A, such as signal and noise qubits. It also resets the *in_use* flags to false and recreates entanglement between pairs in S and N, which just got reset to $|0\rangle$. We only validate that a quantum state is in use, either cloned, yet to be cloned, or already decrypted. If no quantum state is present, we don't allow the user to *uncompute_a*, which is empty because it is redundant. Figure 8 illustrates the quantum circuit uncomputing the quantum state in the index 0.

Figure 7

Quantum circuit after swapping A register 0 and 1 with num_qubits=2 and num_clones=2.

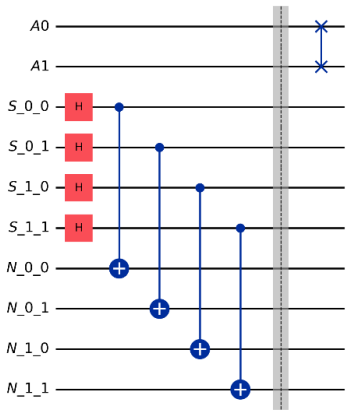
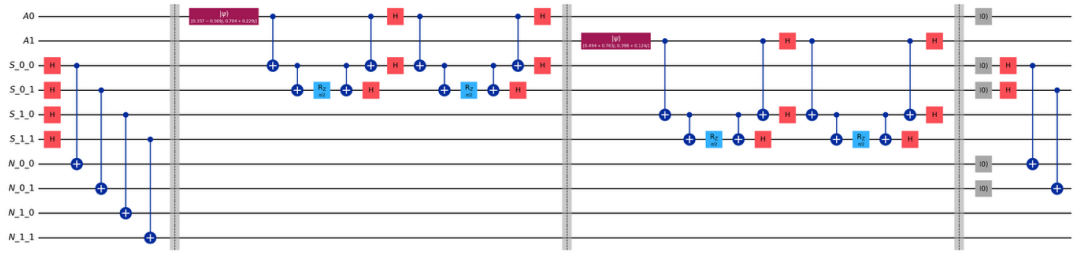


Figure 8

Quantum circuit after storing two quantum states then uncomputing slot 0, showing the reset and entanglement reinitialisation with num_qubits=2 and num_clones=2.



Algorithm 5 Protocol Qubit Management

```
Class Protocol
    procedure SWAP_A(p, q)
        if p < 0 or p ≥ num_qubits or q < 0 or q ≥ num_qubits then
            raise IndexError
        end if
        qc.SWAP(A[p], A[q])
        temp ← A_IN_USE(p)
        val ← A_IN_USE(q)
        A_IN_USE(p, val)
        A_IN_USE(q, temp)
    end procedure

    procedure UNCOMPUTE_A(index)
        if not A_IN_USE(index) then
            raise ValueError
        end if
        qc.reset(A[index])
        A_IN_USE(index, ⊥)
        for i = 0 to num_clones − 1 do
            if S_IN_USE(index, i) or N_IN_USE(index, i) then
                qc.reset(S[index, i])
                qc.reset(N[index, i])
                S_IN_USE(index, i, ⊥)
                N_IN_USE(index, i, ⊥)
            end if
        end for
        for i = 0 to num_clones − 1 do
            qc.H(S[index, i])
            qc.CNOT(S[index, i], N[index, i])
        end for
    end procedure
EndClass
```

4.2.2 QStack Implementation

4.2.2.1 QStack Initialisation

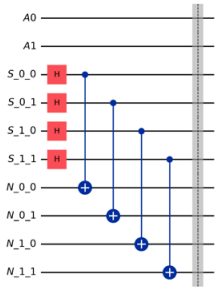
The constructor defined Algorithm 6 initialises the QStack by setting its capacity, *num_qubits*, and redundancy level, *num_clones*, and creates a lookup mapping with a flag to set to false for each slot. This indicates that no quantum state has been stored. The *_size* attribute of QStack is initialised at zero, which serves as the element count and the stack pointer for the next *push*. A protocol instance is constructed with the same *num_qubits* and *num_clones* parameters, which allocates all quantum registers and prepares the entanglement pairs as described in Section 5.2.1.1. Finally, *_get_qc* is set to false, acting as a Boolean lock that will prevent mutations after the circuit is extracted via *generate_circuit*. Figure 9 illustrates the quantum circuit after initialising a QStack with *num_qubits*=2 and *num_clones*=2.

Algorithm 6 QStack Initialisation

```
1:  Class QStack
2:      procedure __INIT__(n_qubits, n_clones)
3:          num_qubits ← n_qubits
4:          num_clones ← n_clones
5:          lookup ← {}
6:          for i = 0 to num_qubits − 1 do
7:              lookup[i] ← {flag : ⊥}
8:          end for
9:          size ← 0
10:         protocol ← Protocol(num_qubits, num_clones)
11:         get_qc ← ⊥
12:     end procedure
13: EndClass
```

Figure 9

QStack initialised with num_qubits=2 and num_clones=2.



4.2.2.2 QStack Utilities

The utility methods in Algorithm 7 provide standard stack query and management functions. The *draw* method delegates to the underlying Qiskit quantum circuit's drawing method. This produces a visual representation of the current circuit state. The *generate_circuit* method returns the underlying quantum circuit from the Protocol and sets the *_get_qc* lock to true, preventing further mutations. This ensures the circuit extracted for compilation and execution remains consistent with the intended sequence of operations. The *size*, *is_empty*, and *is_full* methods return the current element count, whether the stack is empty, and whether it is full, respectively. The clear method resets the QStack by reinitialising *_size* to zero, creating a fresh Protocol instance, unlocking *_get_qc*, and resetting all lookup flags to false. This discards the previous quantum circuit and starts from a clean slate.

Algorithm 7 QStack Utilities

```
1:  Class QStack
2:      function DRAW
3:          return protocol.qc.draw()
4:      end function

5:      function GENERATE_CIRCUIT
6:          get_qc ← ⊤
7:          return PROTOCOL.GET_QC
8:      end function

9:      function SIZE
10:         return size
11:     end function

12:     function IS_EMPTY
13:         return size = 0
14:     end function

15:     function IS_FULL
16:         return size = num_qubits
17:     end function

18:     procedure CLEAR
19:         size ← 0
20:         protocol ← Protocol(num_qubits, num_clones)
21:         get_qc ← ⊥
22:         for i = 0 to num_qubits − 1 do
23:             lookup[i] ← {flag : ⊥}
24:         end for
25:     end procedure
26: EndClass
```

4.2.2.3 QStack Push

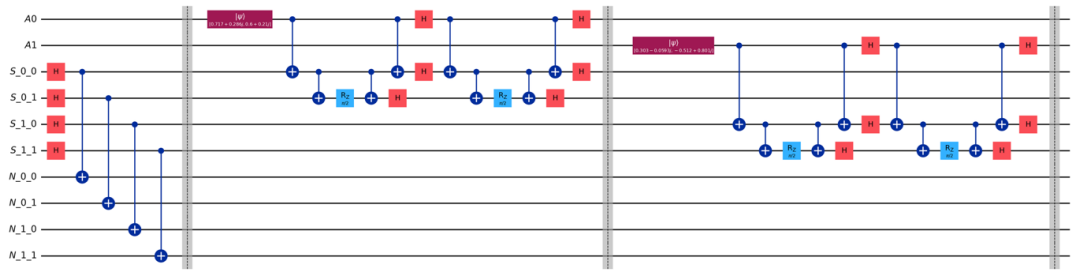
The *push* method defined in Algorithm 8 stores a quantum state onto the top of the stack by using the protocol's *store_qubit* method described in Section 5.2.1.3. The caller must provide a single-qubit quantum circuit containing the quantum state to be stored. The method passes the circuit along with the current *_size* as the slot index, since *_size* always points to the next available position. Once the quantum state gets successfully cloned the lookup flag at that index is set to true, which marks it as the slot occupied, and *_size* is incremented by one to advance the stack pointer. This method enforces four preconditions for a valid *push* call. First, it makes sure that the circuit has not extracted via *generate_circuit*. Second, it makes sure that the users pass a quantum circuit. Third, it makes sure that the passed quantum circuit is single qubit circuit. Finally, it ensures that the pointer has not reached the capacity to avoid overflow. Figure 10 illustrates the quantum circuit after two consecutive push operations on QStack with *num_qubits*=2 and *num_clones*=2.

Algorithm 8 QStack Push

```
1:  Class QStack
2:      procedure PUSH(qc = ∅)
3:          if get_qc then
4:              raise RuntimeError
5:          end if
6:          if qc = ∅ then
7:              raise ValueError
8:          end if
9:          if qc.num_qubits ≠ 1 then
10:             raise ValueError
11:         end if
12:         if size ≥ num_qubits then
13:             raise OverflowError
14:         end if
15:         PROTOCOL.STORE_QUBIT(qc, size)
16:         lookup[size].flag ← ⊤
17:         size ← size + 1
18:     end procedure
19: EndClass
```

Figure 10

Quantum circuit after two consecutive QStack push operations with num_qubits=2 and num_clones=2.



4.2.2.4 QStack Pop

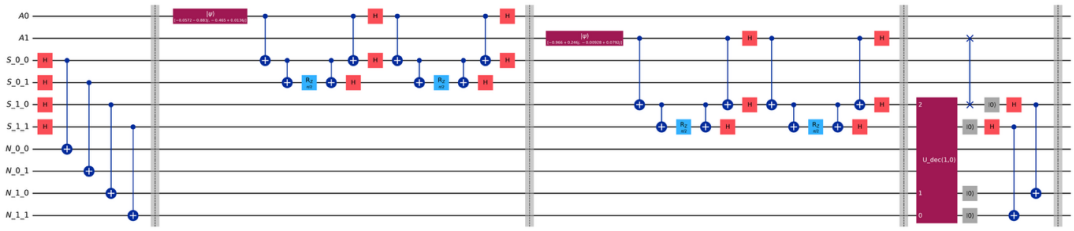
The *pop* method of QStack, defined in Algorithm 9, retrieves the most recently stored quantum state from the stack. To locate the top of the stack, the method scans the lookup table from *_size* – 1downward until it finds a slot with its flag set to true. Once found, it calls the Protocol's *retrieve_qubit* method described in Section 5.2.1.4 with that slot index, *a_index*, and clone index, *c_index*. After successfully retrieving the quantum state to its corresponding data qubit A, we set the lookup flag at that index to false, marking the slot as consumed. Therefore, the next *pop* method cannot attempt to decrypt an already-decrypted quantum state. Moreover, as discussed in section 5.1.2 of the design, the logical size of QStack can never be reduced, because after retrieval, the quantum state remains in the data qubit A, preventing a new *push* from replacing it. This method enforces three preconditions. First, it ensures that the circuit has not been extracted via *generate_circuit*. Second, it ensures the stack is not empty. Third, it ensures the given clone index is within bounds. Figure 11 illustrates the quantum circuit after two pushes, followed by the one *pop* from clone 0 on a QStack with *num_qubits*=2 and *num_clones*=2.

Algorithm 9 QStack Pop

```
Class QStack
    procedure POP(c_index = 0)
        if get_qc then
            raise RuntimeError
        end if
        if IS_EMPTY then
            raise IndexError
        end if
        if c_index < 0 or c_index ≥ num_clones then
            raise ValueError
        end if
        idx ← 0
        for i = size − 1 downto 0 do
            if lookup[i].flag then
                idx ← i
                break
            end if
        end for
        PROTOCOL.RETRIEVE_QUBIT(idx, c_index)
        lookup[idx].flag ← ⊥
    end procedure
EndClass
```

Figure 11

Quantum circuit after two QStack push operations followed by one pop from clone 0, with num_qubits=2 and num_clones=2.



4.2.3 QArray Implementation

4.2.3.1 QArray Initialisation

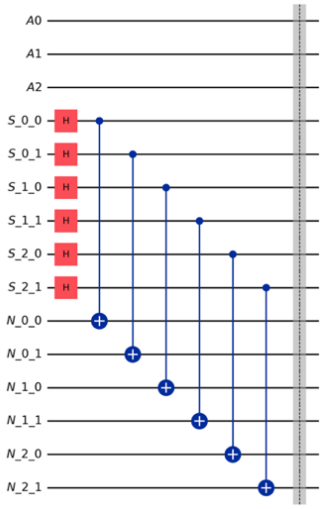
The constructor defined in Algorithm 10 initialises the QArray by setting its capacity, *num_qubits*, and redundancy level, *num_clones*, then creating a lookup mapping with a status string set to empty for each slot. Unlike QStack's binary flag, QArray uses three status values, which are empty, set, and get. This design choice was made to distinguish in our array data structure between slots that have never been used and those that have been decrypted to retrieve the stored quantum state. The *_size* attribute, which is used to track the number of currently stored elements, is initialised to zero, and the *_tail* attribute, which is used to track the highest occupied index plus one, is also initialised to zero. These two attributes are maintained independently because the user can set elements at arbitrary indices, therefore, potentially leaving intermediate slots empty. A protocol instance defined in Section 5.2.1.1 is constructed with the parameters of QArray's capacity and redundancy level. Finally, *_get_qc* is set to false, acting as a Boolean lock that will prevent mutations after the circuit is extracted via *generate_circuit*. Figure 12 illustrates the quantum circuit after initialising a QArray with *num_qubits*=3 and *num_clones*=2.

Algorithm 10 QArray Initialisation

```
1:  Class QArray
2:      procedure __INIT__(n_qubits, n_clones)
3:          num_qubits ← n_qubits
4:          num_clones ← n_clones
5:          lookup ← {}
6:          for i = 0 to num_qubits − 1 do
7:              lookup[i] ← {status : ""}
8:          end for
9:          size ← 0
10:         tail ← 0
11:         protocol ← Protocol(num_qubits, num_clones)
12:         get_qc ← ⊥
13:     end procedure
14: EndClass
```

Figure 12

QArray initialised with num_qubits=3 and num_clones=2.



4.2.3.2 QArray Utilities

The utility methods in Algorithm 11 provide standard array query and management functions. The *draw* method delegates to the underlying Qiskit quantum circuit's drawing method. This produces a visual representation of the current circuit state. The *generate_circuit* method returns the underlying quantum circuit from the Protocol and sets the *_get_qc* lock to true, preventing further mutations. This ensures the circuit extracted for compilation and execution remains consistent with the intended sequence of operations. The *size*, *is_empty*, and *is_full* methods return the current element count, whether the array is empty, and whether it is full, respectively. The clear method resets the QArray by reinitialising *_size* and *_tail* to zero, creating a fresh Protocol instance, unlocking *_get_qc*, and resetting all lookup flags to an empty string. This discards the previous quantum circuit and starts from a clean slate.

Algorithm 11 QArray Utilities

```
1:  Class QArray
2:      function DRAW
3:          return protocol.qc.draw()
4:      end function

5:      function GENERATE_CIRCUIT
6:          get_qc ← ⊤
7:          return protocol.qc
8:      end function

9:      function SIZE
10:         return size
11:     end function

12:     function IS_EMPTY
13:         return size = 0
14:     end function

15:     function IS_FULL
16:         return size = num_qubits
17:     end function

18:     procedure CLEAR
19:         size ← 0
20:         tail ← 0
21:         protocol ← Protocol(num_qubits, num_clones)
22:         get_qc ← ⊥
23:         for i = 0 to num_qubits − 1 do
24:             lookup[i] ← {status : ""}
25:         end for
26:     end procedure
27: EndClass
```

4.2.3.3 QArray Set

The *set* method defined in Algorithm stores a quantum state at a specific index in the QArray by using the protocol's store_qubit method described in Section 5.2.1.3. The caller must provide both an index and a single-qubit quantum circuit containing the quantum state. The method passes the circuit along with the index to the protocol's *store_qubit* function. Once the quantum state gets successfully cloned, the lookup flag at that index is set to "set", which marks it as the slot occupied but not retrieved yet, *_size*

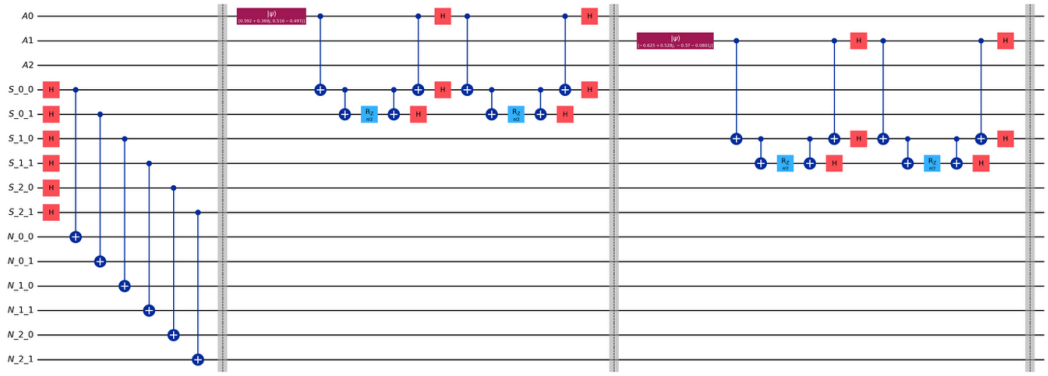
is incremented by one, and *_tail* is updated to the maximum of its current value or index plus one, since the user can set elements at arbitrary indices. This method enforces five preconditions for a valid set call. First, it makes sure that the circuit has not been extracted via *generate_circuit*. Second, it makes sure that the users pass an index and a quantum circuit. Third, it makes sure that the passed quantum circuit is a single-qubit circuit. Fourth, it ensures that the passed index is within the array's bounds. Finally, it ensures that the passed index does not already have a quantum state stored or retrieved. Figure 13 illustrates the quantum circuit after setting two quantum states at index 0 and 1 in a QArray with *num_qubits*=3 and *num_clones*=2.

Algorithm 12 QArray Set

```
1:  Class QArray
2:      procedure SET(index = ∅, qc = ∅)
3:          if get_qc then
4:              raise RuntimeError
5:          end if
6:          if index = ∅ or qc = ∅ then
7:              raise ValueError
8:          end if
9:          if qc.num_qubits ≠ 1 then
10:             raise ValueError
11:         end if
12:         if index < 0 or index ≥ num_qubits then
13:             raise IndexError
14:         end if
15:         if lookup[index].status = "set" or lookup[index].status = "get" then
16:             raise IndexError
17:         end if
18:         PROTOCOL.STORE_QUBIT(qc, index)
19:         lookup[index].status ← "set"
20:         size ← size + 1
21:         tail ← max(tail, index + 1)
22:     end procedure
23: EndClass
```

Figure 13

Quantum circuit after setting two quantum states at index 0 and 1 in a QArray with num_qubits=3 and num_clones=2.



4.2.3.4 QArray Get

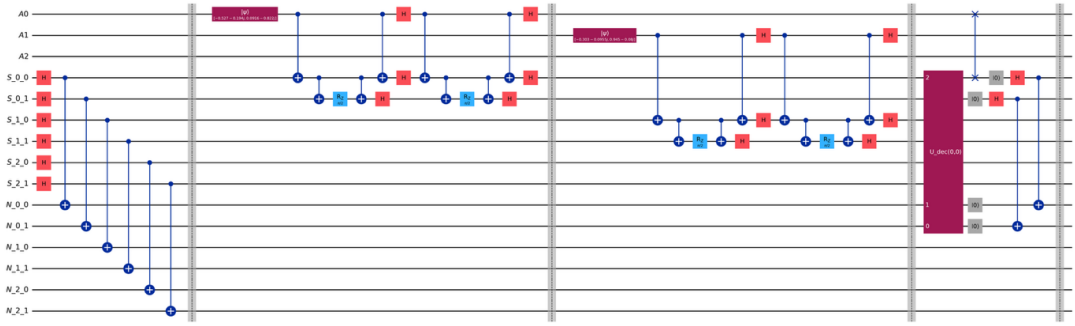
The *get* method of QArray, defined in Algorithm 13, retrieves a quantum state from a specified index in the QArray by using the protocol's *retrieve_qubit* function defined in Section 5.2.1.4. The caller must provide an *a_index* specifying which slot to retrieve from and a *c_index* specifying which of the n encrypted clones to decrypt. This method passes the *a_index* and *c_index* to the protocol's *retrieve_qubit* function. Once the quantum state is successfully retrieved, the lookup status at that index is set to "get", marking the slot as unavailable to avoid overwriting with a different quantum state. This is done because the quantum state is returned to the corresponding qubit in the A register, which may be used for further computations, as discussed in the design of our framework. The method enforces three preconditions. First, it makes sure that the circuit has not been extracted via *generate_circuit*. Second, it ensures that users pass an index to retrieve. Third, it ensures that the passed indexes, *a_index* and *c_index*, are within the bounds of the array and the clone ranges, respectively. Figure 14 illustrates the quantum circuit after setting two quantum states, then retrieving from index 0 clone 0 in a QArray with *num_qubits*=3 and *num_clones*=2.

Algorithm 13 QArray Get

```
Class QArray
    procedure GET(a_index = ∅, c_index = 0)
        if get_qc then
            raise RuntimeError
        end if
        if a_index = ∅ then
            raise ValueError
        end if
        if a_index < 0 or a_index ≥ num_qubits then
            raise IndexError
        end if
        if c_index < 0 or c_index ≥ num_clones then
            raise IndexError
        end if
        PROTOCOL.RETRIEVE_QUBIT(a_index, c_index)
        lookup[a_index].status ← "get"
    end procedure
EndClass
```

Figure 14

Quantum circuit after setting two quantum states then retrieving from index 0 clone 0, with num_qubits=3 and num_clones=2.

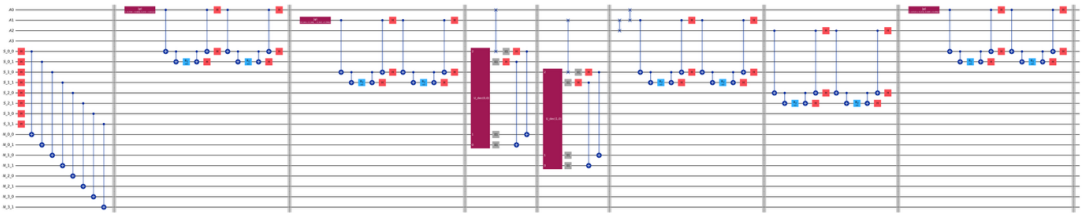


4.2.3.5 QArray Insert

The *insert* method defined in Algorithm 14 stores a quantum state at a specified index by first shifting all existing elements to the right to make room, then applying encrypted cloning on the new state at the target index. Since quantum states cannot be copied, the shifting process requires a retrieve-swap-store cycle. First, all the slots from the insertion index to the tail that have a status of "set" are retrieved via the protocol's *retrieve_qubit* function back to their corresponding qubit A registers. Then, SWAP gates are applied from the tail downward to the insertion index, shifting each qubit A one position to the right, with the lookup table statuses swapped in tandem. After shifting, all the slots that still have a status of "set" are restored via the protocol's *store_qubit* function without passing any circuit, since the quantum states are already in the qubit A registers from the prior swaps. This will not restore quantum states that have already been retrieved. Finally, the new quantum state is stored at the target index using the protocol's *store_qubit* function. Additionally, the lookup status is set to "set," and both *_size* and *_tail* are incremented by 1. This method enforces five preconditions. First, it makes sure that the circuit has not been extracted via *generate_circuit*. Second, it makes sure that the users pass an index and a quantum circuit. Third, it makes sure that the passed quantum circuit is a single-qubit circuit. Fourth, it ensures that the passed index is within the array's bounds. Finally, it ensures that the size does not exceed the array's capacity. Figure 15 illustrates the quantum circuit after setting two quantum states, then inserting a new state at index 0 in a QArray with *num_qubits*=4 and *num_clones*=2.

Figure 15

Quantum circuit after setting two quantum states then inserting a new state at index 0, with num_qubits=4 and num_clones=2



Algorithm 14 QArray Insert

```
1:  Class QArray
2:      procedure INSERT(index, qc = ∅)
3:          if get_qc then
4:              raise RuntimeError
5:          end if
6:          if index = ∅ or qc = ∅ then
7:              raise ValueError
8:          end if
9:          if qc.num_qubits ≠ 1 then
10:             raise ValueError
11:         end if
12:         if index < 0 or index ≥ num_qubits then
13:             raise IndexError
14:         end if
15:         if size ≥ num_qubits then
16:             raise IndexError
17:         end if
18:         for i = index to tail − 1 do
19:             if lookup[i].status = "set" then
20:                 PROTOCOL.RETRIEVE_QUBIT(i, 0)
21:             end if
22:         end for
23:         for i = tail − 1 downto index do
24:             PROTOCOL.SWAP_A(i, i + 1)
25:             temp ← lookup[i].status
26:             lookup[i].status ← lookup[i + 1].status
27:             lookup[i + 1].status ← temp
28:         end for
29:         for i = index + 1 to tail do
30:             if lookup[i].status = "set" then
31:                 PROTOCOL.STORE_QUBIT(∅, i)
32:             end if
33:         end for
34:         PROTOCOL.STORE_QUBIT(qc, index)
35:         lookup[index].status ← "set"
36:         size ← size + 1
37:         tail ← tail + 1
38:     end procedure
39: EndClass
```

4.2.3.6 QArray Append

The *append* method defined in Algorithm 15 stores a quantum state at the tail of the QArray, making it functionally equivalent to calling *set(_tail, qc)*. The caller must provide a single-qubit quantum circuit containing the quantum state to be stored. The method passes the circuit and the current tail value to the protocol's *store_qubit* function. After

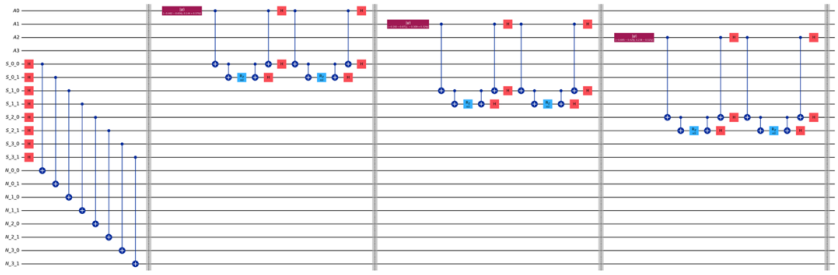
the successful encrypted cloning, the lookup status at the tail is set to “set”, and both *_size* and *_tail* are incremented. Since append always targets the next unoccupied position at the end of the array, no shifting or retrieve-swap-store cycles are required. This method enforces four preconditions for a valid append call. First, it ensures that the circuit has not been extracted via *generate_circuit*. Second, it ensures that users pass a quantum circuit. Third, it makes sure that the passed quantum circuit is a single-qubit circuit. Finally, it ensures the tail is not out of bounds. Figure 16 illustrates the quantum circuit after setting two quantum states, then append a new state in a QArray with *num_qubits*=4 and *num_clones*=2.

Algorithm 15 QArray Append

```
1:  Class QArray
2:      procedure APPEND(qc = ∅)
3:          if get_qc then
4:              raise RuntimeError
5:          end if
6:          if qc = ∅ then
7:              raise ValueError
8:          end if
9:          if qc.num_qubits ≠ 1 then
10:             raise ValueError
11:         end if
12:         if tail ≥ num_qubits then
13:             raise IndexError
14:         end if
15:         PROTOCOL.STORE_QUBIT(qc, tail)
16:         lookup[tail].status ← “set”
17:         size ← size + 1
18:         tail ← tail + 1
19:     end procedure
20: EndClass
```

Figure 16

Quantum circuit after setting two quantum states then appending a third, with num_qubits=4 and num_clones=2.

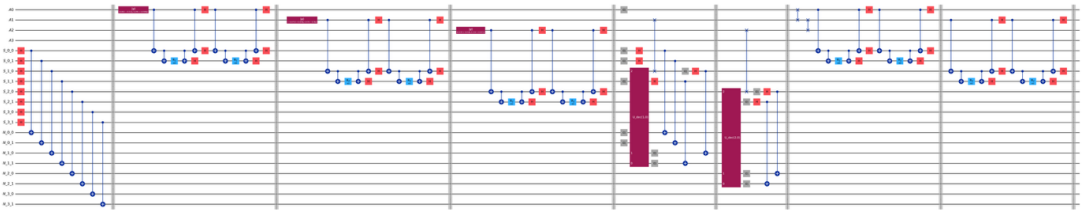


4.2.3.7 QArray Remove

The *remove* method of QArray, defined in Algorithm 16, discards the quantum state at a specified index and shifts all elements to the right of it one position to the left to fill the gap. The target slot is first uncomputed using the protocol's *uncompute_a* function defined in Algorithm 5. This resets qubit A along with its signal and noise qubits and then reinitialises their Bell pairs and marks the lookup status as an empty string. Similar to QArray's *insert* method, *remove* shifts the index to the left using the retrieve-swap-store cycle. Both *_size* and *_tail* are decremented by one at the end. This method enforces three preconditions for a valid remove call. First, it makes sure that the circuit has not been extracted via *generate_circuit*. Second, it makes sure that the users pass an index to target. Finally, it ensures that the given index is within the bounds of the array's capacity. Figure 17 illustrates the quantum circuit after setting three quantum states, then remove the state in index 0 a QArray with *num_qubits*=4 and *num_clones*=2.

Figure 17

Quantum circuit after setting three quantum states then removing index 0, with num_qubits=4 and num_clones=2.



Algorithm 16 QArray Remove

```
1:  Class QArray
2:      procedure REMOVE(index = ∅)
3:          if get_qc then
4:              raise RuntimeError
5:          end if
6:          if index = ∅ then
7:              raise ValueError
8:          end if
9:          if index < 0 or index ≥ tail then
10:             raise IndexError
11:         end if
12:         PROTOCOL.UNCOMPUTE_A(index)
13:         lookup[index].status ← ""
14:         for i = index + 1 to tail − 1 do
15:             if lookup[i].status = "set" then
16:                 PROTOCOL.RETRIEVE_QUBIT(i, 0)
17:             end if
18:         end for
19:         for i = index to tail − 2 do
20:             PROTOCOL.SWAP_A(i, i + 1)
21:             temp ← lookup[i].status
22:             lookup[i].status ← lookup[i + 1].status
23:             lookup[i + 1].status ← temp
24:         end for
25:         for i = index to tail − 2 do
26:             if lookup[i].status = "set" then
27:                 PROTOCOL.STORE_QUBIT(∅, i)
28:             end if
29:         end for
30:         size ← size − 1
31:         tail ← tail − 1
32:     end procedure
33: EndClass
```

4.2.3.8 QArray Reverse

The *reverse* method defined in Algorithm 17 reverses the order of all stored quantum states in the QArray. Since quantum states cannot be copied, this method also requires the retrieve-swap-store cycle, similar to the *insert* and *remove* methods. The only precondition for this method is that the circuit has not yet been generated with

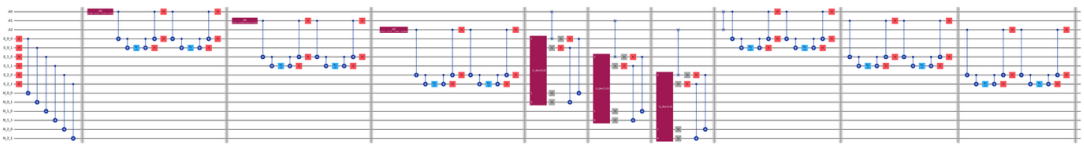
generate_circuit. Figure 18 illustrates the quantum circuit after setting three quantum states, then reverse a QArray with *num_qubits*=3 and *num_clones*=2.

Algorithm 17 QArray Reverse

```
Class QArray
    procedure REVERSE
        if get_qc then
            raise RuntimeError
        end if
        for i = 0 to tail − 1 do
            if lookup[i].status = "set" then
                PROTOCOL.RETRIEVE_QUBIT(i, 0)
            end if
        end for
        for i = 0 to ⌊tail/2⌋ − 1 do
            j ← tail − 1 − i
            PROTOCOL.SWAP_A(i, j)
            temp ← lookup[i].status
            lookup[i].status ← lookup[j].status
            lookup[j].status ← temp
        end for
        for i = 0 to tail − 1 do
            if lookup[i].status = "set" then
                PROTOCOL.STORE_QUBIT(∅, i)
            end if
        end for
    end procedure
EndClass
```

Figure 18

Quantum circuit after setting three quantum states then reversing the array, with num_qubits=3 and num_clones=2.



Note. Figure 4 to 18 full resolution files can be accessed at https://github.com/sibishan/encrypted-cloning-data-structures/tree/main/presentation/out

UTS

5. Experimental Analysis

5.1 Experimental Environment

All experiments were conducted through classical simulation of quantum circuits. The quantum circuits are constructed using the Qiskit library in Python. Therefore, our framework requires Python 3.14.3, Qiskit 2.3.1, and NumPy 2.4.3 to construct quantum circuits for the encrypted cloning protocol and the data structures that encapsulate it. Moreover, to run experiments in our framework, we require Qiskit Aer 0.17.2. We simulate the quantum circuits under ideal, error-free, and noisy configurations. We use Qiskit Aer's statevector simulator for ideal experiments and its density-matrix simulator for noisy experiments. The noisy configuration allows us to simulate currently available quantum devices, which are highly prone to errors during computation. We selected a model based on the current state-of-the-art IBM quantum computer, Heron R2. Heron R2 has single-qubit depolarising error $\rho_1 = 10^{-4}$ and two-qubit depolarising error $\rho_2 = 5 \times 10^{-3}$. Furthermore, libraries such as os, matplotlib, json, time, datetime, and Jupyter Notebook are also used during experiments to store and manipulate data obtained from the simulations. To guarantee reproducibility of our results, input states were generated as Haar-random single-qubit states using Qiskit's *random_statevector* function, with distinct deterministic seeds per trial. This approach provides uniform coverage of all possible quantum states, helping us avoid bias toward any particular basis state.

A MacBook Pro with an M2 Pro chip and 16GB RAM is used to run the experiments. Quantum simulation is bottlenecked by memory requirements, which grow exponentially with the number of qubits in the circuit. It is estimated that simulations with 30 qubits would require 16 GB of RAM, and 40 qubits would require 16 TB of RAM (Google Quantum AI, 2024). Therefore, we limited our circuits to 30 qubits. Distributed quantum simulation frameworks such as Das and DiAdamo (2023) and Sreraman M (2025) exist but remain unmaintained and unpublished, so all experiments were conducted on a single-process Qiskit Aer simulator. This limits our evaluation to single-core quantum storage rather than the multi-cloud deployment envisioned by Yamaguchi and Kempf (2026).

5.2 Evaluation Criteria

We evaluate our framework across three dimensions, which are correctness, encryption security, and resource scalability. Together, these dimensions assess whether the framework faithfully implements the Yamaguchi and Kempf (2026) encrypted cloning protocol and whether the data structure abstractions introduce acceptable overhead.

Correctness is measured by decryption fidelity. Nielsen and Chuang (2010) define that the fidelity between a pure input state $|\psi\rangle$ and the recovered reduced density matrix ρ_A can be calculated as follows

$$F(|\psi\rangle, \rho_A) = \langle\psi| \rho_A |\psi\rangle, \quad (7)$$

This quantifies how closely the retrieved quantum state matches the original input state. Qiskit provides the *state_fidelity* function to perform a fidelity calculation. Under ideal simulation, the protocol guarantees a fidelity of 1.0 (Yamaguchi & Kempf, 2026). In noisy simulations, where we simulate IBM Heron R2 hardware specifications, we can expect fidelity to degrade due to depolarising errors accumulated through the circuit's gates; particularly, two-qubit gates have a higher error rate than single-qubit gates (Google Quantum AI, 2024).

After the encryption, the Yamaguchi and Kempf's (2026) protocol theoretically guarantees that every individual qubit, data qubits A, signal qubits S, and noise qubits N, should be in a maximally mixed state with a purity of 0.5. This means no single qubit leaks any information about the stored quantum state. Nielsen and Chuang (2010) define the purity of a single-qubit reduced density matrix ρ as follows

$$\gamma(\rho) = \mathrm{Tr}(\rho^2), \quad (5)$$

Here ρ can be obtained by tracing out all other qubits from the full system state after encryption, expect the single qubit in question, to find its purity.

Resource scalability is measured by four metrics. First, total qubit count, which should follow the theoretical formula $m(2n + 1)$, where m is the number of stored qubits, and n is the number of clones. Second, a two-qubit gate, such as CZ count, which is the dominant source of error on current quantum devices, such as IBM Heron R2. Third, we extract the total gate count. Fourth, we extract circuit depth, which determines the decoherence level accumulated during the execution. Additionally, we record the time

consumption for each core action of our framework, breaking it down into encryption, decryption, and simulation time to assess practical overhead.

5.3 Validation Study

Rather than a traditional ablation, we validate each layer of the framework by testing the above-mentioned metrics across the following configurations. To measure fidelity, we utilise the Protocol containing a stored qubit (m = 1) and n = 2, 3, 4, 5 clones, with two decryption modes (first and last clones) for ideal simulations and one decryption mode (first clone) for noisy simulations, across 100 Haar-random input states per configuration. This yields 800 ideal trials and 400 noisy trials. To measure the quality of encryption, we evaluate n = 2, 3, 4, and 5 clones with 100 Haar-random states per configuration, yielding 400 trials. To measure resource scaling, we conduct two sweeps. The first sweep is for clones, where we set m = 1 (the stored qubit) and n = 2, 3, 4, 5, 6, 7, 8. The second sweep is for stored qubits, where we set m = 1, 2, 3, or 4 stored qubits and n = 2 or 3 clones. Each sweep benchmarks four framework components, which are Protocol store and retrieve, QArray set and get, QArray append and get, and QStack push and pop. Our largest tested configuration, with m = 4 stored qubits and n = 3 clones, requires 28 qubits, approaching the practical memory ceiling of our hardware.

We first evaluate the Protocol layer in isolation for the fidelity check. In an ideal quantum environment, the protocol maintains the fidelity of the stored qubit at 1.0. This is evident in Figure 19, where all 800 ideal-simulation trials achieved a fidelity of 1.0. This confirms our implementation correctly reproduces the protocol's theoretical guarantee of perfect state recovery. Moreover, as shown in Figure 20, the encryption quality benchmark indicates that all individual qubits have a purity of exactly 0.5 across all 400 experimental trials.

We next evaluate whether our data structure operates as intended. Exhaustive testing was performed on our data structures, QStack and QArray, including edge cases. This testing required us to manually verify the quantum circuit produced by each test case. The reader can run the Python scripts available at github.com/sibishan/encrypted-cloning-data-structures/tree/main/tests to compare the terminal output with the expected outcomes annotated in the scripts. In addition, we can analyse the resource

scaling factor for the primary storage and retrieval functions, such as push and pop for QStack and set and get for QArray, using varying numbers of stored qubits and clones. It is evident that these basic operations produce identical qubit counts, CZ gate counts, and circuit depth to the raw Protocol at the same m and n configurations as shown in Figures 21 and 22.

Figure 19

Protocol fidelity checks on a stored quantum state with varying number of clones.

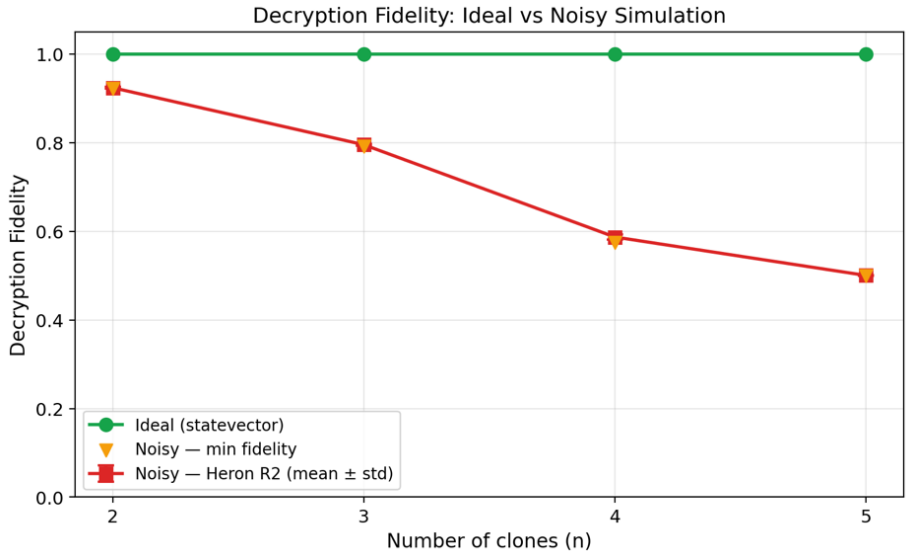


Figure 20

Assessing the quality of encryption using purity of data qubit A, signal qubit S, and Noise qubit S.

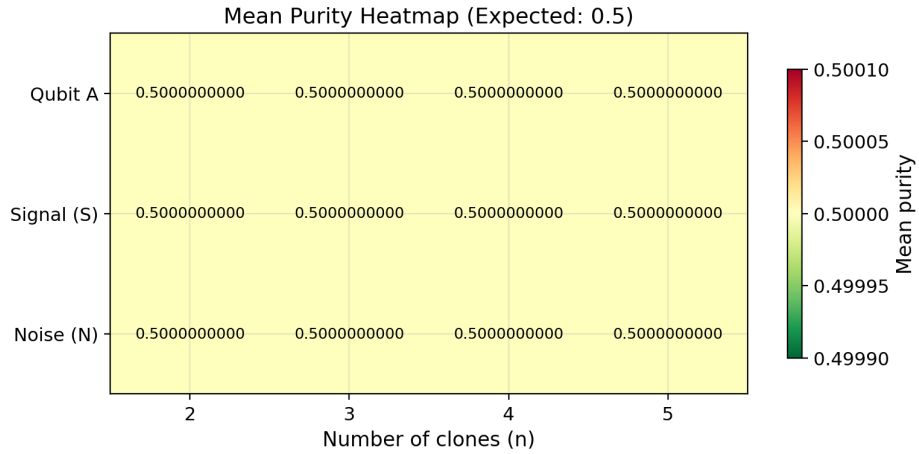


Figure 21

Resource scaling assessed with a stored qubit and varying n = 2, 3, 4, 5, 6, 7, 8 clones.

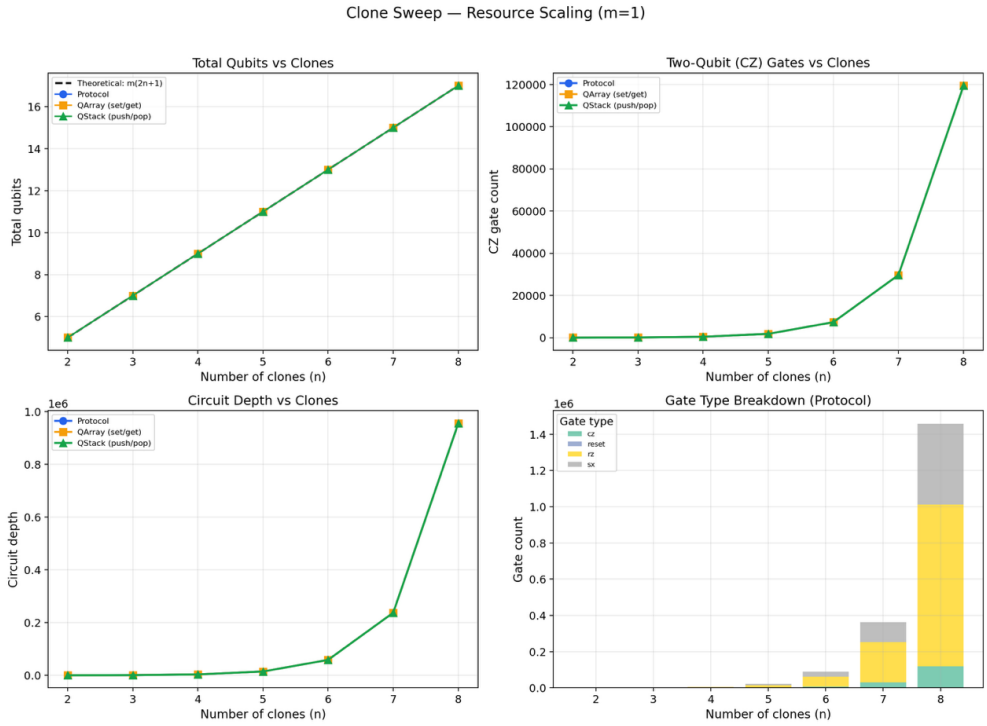
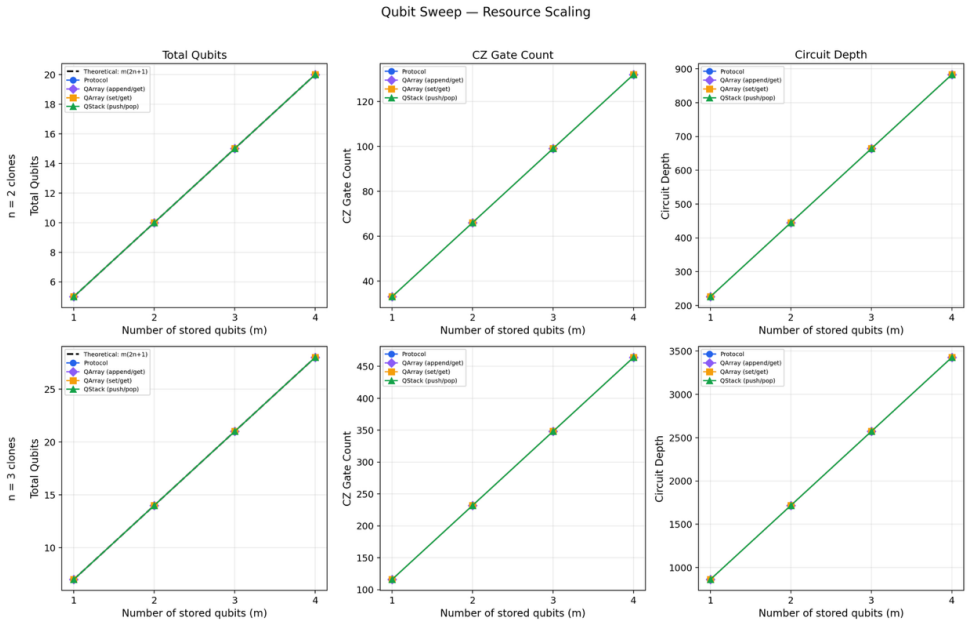


Figure 22

Resource scaling assessed with varying m = 1, 2, 3, 4 stored qubit and varying n = 2, 3 clones.

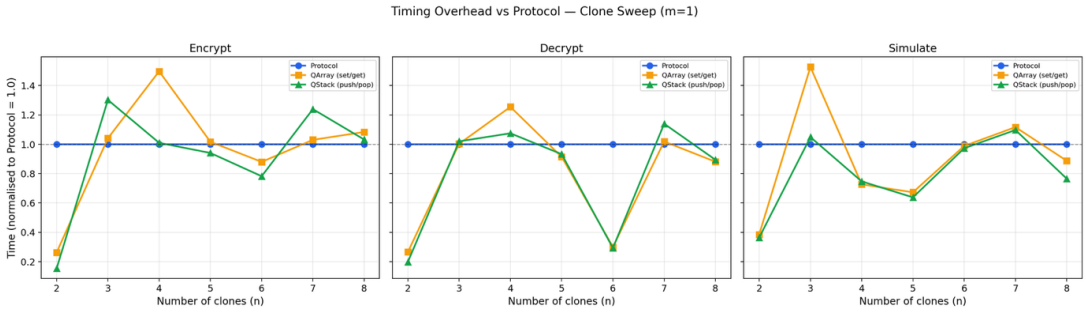


5.4 Performance Analysis

We further analysed our data structures regarding the time consumption per action, such as encryption, decryption, and simulation, against the normalised time consumption of the Protocol. In Figure 23, we observe fluctuations in time consumption as we store one quantum state with varying numbers of clones. Since these operations take milliseconds to execute, even a tiny variation in system load could produce large ratios. Therefore, in this case, we can rule out these fluctuations and conclude that the overhead is negligible. However, we can observe that in Figure 24, as the number of stored qubits m and clones n grow, the classical bookkeeping of our data structures introduces overhead when encryption and decryption operations are performed, but the simulation time remains unchanged, ignoring fluctuations caused by system load, because all Protocol, QArray, and QStack produce identical circuits. Therefore, the overhead is in circuit construction time, not circuit execution time.

Figure 23

Timing overhead of QArray and QStack operations normalised to Protocol (= 1.0) for a single stored qubit with varying n = 2, 3, 4, 5, 6, 7, 8 clones, broken down by encryption, decryption, and simulation phases.



Current quantum devices suffer from noise during single-qubit and two-qubit gate operations; therefore, we can expect the fidelity of the recovered state to degrade as the number of clones increases. Figure 19 illustrates how the fidelity degrades as the number of clones increases. This is because each operation performed in the quantum circuit accumulates error as we keep performing operations. Furthermore, Figure 25 illustrates the distribution of fidelity of the recovered state at each number of clones obtained from our 400 trials.

Figure 24

Timing overhead of QArray and QStack operations normalised to Protocol (= 1.0) with varying m = 1, 2, 3, 4 stored qubits at n = 2 clones (top row) and n = 3 clones (bottom row), broken down by encryption, decryption, and simulation phases.

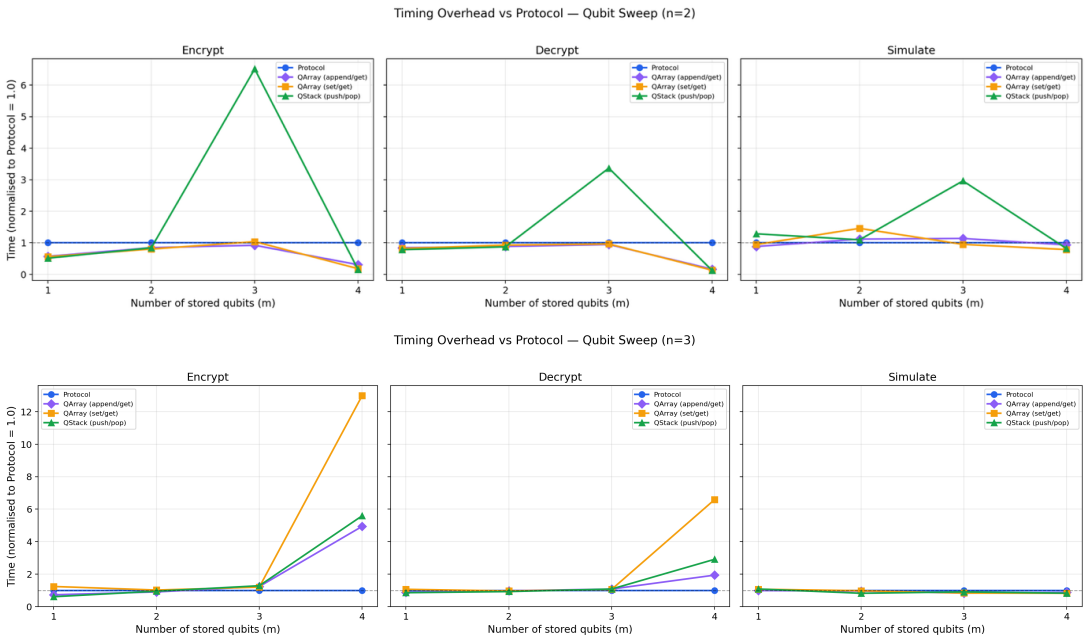
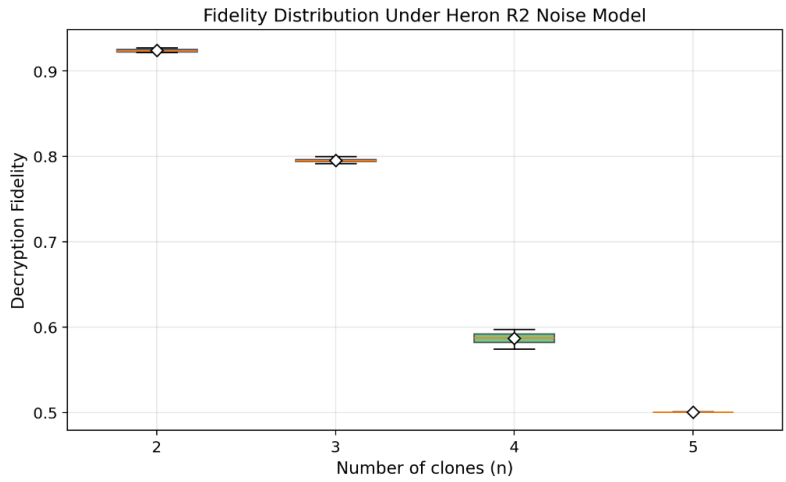


Figure 25

Distribution of decryption fidelity under the Heron R2 noise model across 100 Haar-random input states per configuration, for n = 2, 3, 4, 5 clones with m = 1 stored qubit.

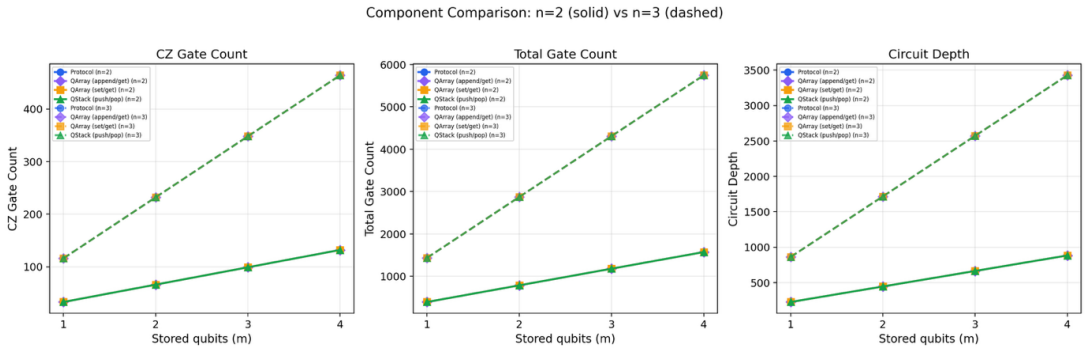


When we consider the plots in Figures 21 and 22, we deduce that the resources such as qubits, gate count, and circuit depth grow linearly as the number of stored qubits

increases for a constant number of clones. On the other hand, when the number of data qubits stays constant, the gate count and circuit depth grow exponentially while the number of qubits in the circuit scales linearly. This exponential growth in gate count is the main factor contributing to the fidelity degradation shown in Figures 19 and 25. Even though the gate count and circuit depth grow linearly for a constant number of clones. Figure 26 illustrates the gradient difference between two different constant numbers of clones as the number of stored qubits increases.

Figure 26

Comparison of CZ gate count, total gate count, and circuit depth between n = 2 (solid) and n = 3 (dashed) clones as the number of stored qubits m increases from 1 to 4.



5.5 Comparison against the Theoretical and Experimental Baselines

Yamaguchi and Kempf's (2026) protocol is purely theoretical and lacks implementation, while Yamaguchi et al. (2026) provide a demonstration on IBM Heron R2 hardware but do not package the protocol as a reusable data structure abstraction. Our framework bridges the gap between these by deriving executable quantum circuits from high-level data structure operations and validating them against both baselines. In Figure 19, we observed that the ideal simulation achieves a fidelity of 1.0 across all 800 trials, matching the theoretical guarantee of Yamaguchi and Kempf (2026). Moreover, our total qubit count adheres to the $m(2n + 1)$ scaling, which was evident in Figures 21 and 22.

Our noisy-simulation fidelities qualitatively reproduce the degradation trend observed by Yamaguchi et al. (2026) on real quantum hardware, IBM Kingston, with fidelity decaying gradually as n increases. For instance, at n=2 we observe fidelity ≈ 0.92 versus their 0.823 ± 0.004, while at n=5 we observe ≈ 0.5 versus their 0.679 ± 0.005. The exact value differs

because our depolarising noise model ($p_1 = 10^{-4}$, $p_2 = 5 \times 10^{-3}$) does not capture real hardware effects such as crosstalk, coherent errors, readout noise, calibration drift, and SWAP overhead. However, both our noisy simulation and Yamaguchi et al. (2026) hardware results agree on the underlying mechanism, where fidelity degradation is dominated by accumulated gate error rather than any intrinsic instability with clone count. This positions our framework as a validated software layer that complements rather than duplicates either baseline, enabling researchers to build encrypted quantum storage applications without manually constructing protocol circuits.

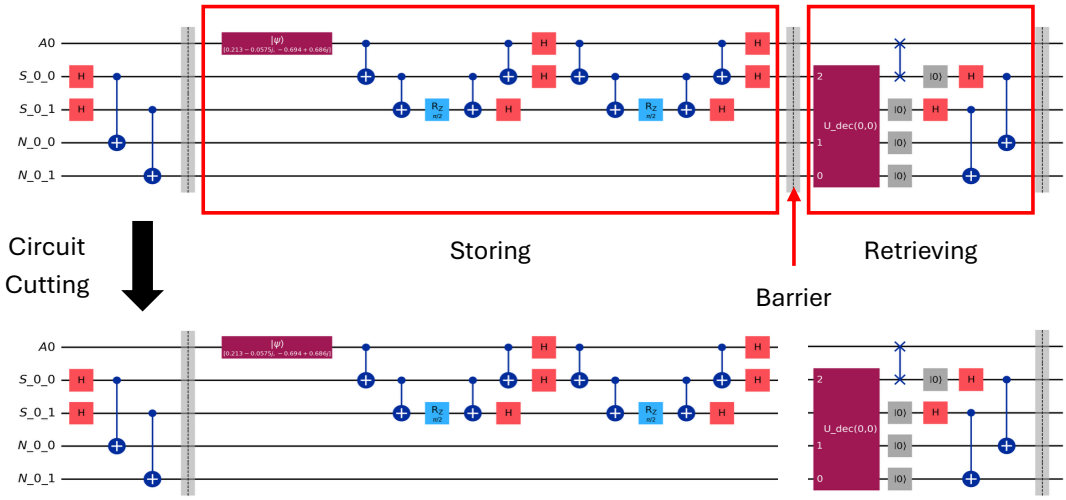
6. Discussion

The quantum circuits presented throughout Section 4.2 treat storage and retrieval as a single monolithic circuit executed in one run. While this representation is useful for analysis and simulation, it does not reflect how an end user would actually interact with quantum-encrypted cloud storage. In practice, a user of this framework would expect to store quantum states and retrieve them at a later time. This is what Yamaguchi and Kempf's (2026) encrypted cloning protocol envisions. Therefore, to enable our framework for use in real-world scenarios, we propose using a circuit-cutting strategy. To make this work, the user must generate a circuit that stores the quantum states and their corresponding retrieval sequences as a single circuit using any of our data structures. As visualised in Figures 4 to 18, a quantum circuit can be considered as a piece of paper that can be vertically cut into pieces. Figure 27 illustrates a circuit cut into two pieces, where the first piece can store a quantum state and the second can retrieve it.

In our protocol implementation, we included barriers to be applied in the quantum circuit after each store_qubit and retrieve_qubit operation. End users can consider these barriers as points where they can cut the circuit into two pieces. Algorithm 18 details the circuit-cutting utility function we implemented for the end user.

Figure 27

Quantum circuit cutting.



Algorithm 18 Circuit Cutting Utility

```
function CUT_CIRCUIT(qc, idx)
    qc_1 ← QuantumCircuit(qc.qregs, qc.cregs)
    qc_2 ← QuantumCircuit(qc.qregs, qc.cregs)
    barrier_count ← 0
    target ← qc_1
    for each instr in qc.data do
        if instr.operation.name = "barrier" then
            if barrier_count = idx then
                target ← qc_2
            end if
            barrier_count ← barrier_count + 1
            continue
        end if
        target.APPEND(instr.operation, instr.qubits, instr.clbits)
    end for
    if target is qc_1 then
        raise ValueError
    end if
    return qc_1, qc_2
end function
```

When the user wants to cut a quantum circuit they generated into several pieces, they can recursively call Algorithm 18, assuming they already analysed the whole circuit and recorded barrier indexes they want to cut at. Our future work may consider a simpler

circuit-cutting procedure to address scaling issues that may arise when many operations are performed on the data structure.

However, the viability of using our framework, which encapsulates Yamaguchi and Kempf's (2026) encrypted cloning protocol, is currently limited by the currently available quantum hardware. The first issue is decoherence in Noisy Intermediate-Scale Quantum (NISQ) computers, which is the current era of quantum computers. Even the most advanced superconducting devices, including the IBM Heron R2 model we adopted in Section 5, maintain quantum coherence for only 50-150 microseconds (AbuGhanem, 2025). Once the quantum computer reaches its coherent time threshold, it will start losing fidelity of its quantum state, as we observed in our noisy simulations. This lifespan is far shorter than what a cloud storage user would expect for data availability. Until quantum memory technologies with substantially longer coherence times mature beyond laboratory demonstrations, the encrypted clones cannot persist long enough to support practical cloud-storage use cases. Furthermore, the multi-cloud storage application proposed by Yamaguchi and Kempf (2026) inherently requires distributed quantum computers. Distributed quantum computing is still a very recent research area, and the scale currently under development is small (Caleffi et al., 2024). Therefore, our framework only considers a single-core quantum computer to store the user's quantum state. Although our framework could be extended to support distributed quantum computers, further development is needed to do so, which depends on the maturation of distributed quantum compilers and inter-QPU networking infrastructure. Our framework should therefore be understood as a software abstraction whose operational deployment depends on hardware capabilities that do not yet exist, rather than a system that can be deployed on today's devices.

7. Conclusion

This study presented the first software framework that abstracts Yamaguchi and Kempf's (2026) encrypted cloning protocol behind familiar data structure interfaces. The Protocol class encapsulates the Qiskit quantum operations of the encrypted cloning protocol. The data structures QArray and QStack provide indexed and last-in-first-out access, respectively. Under the hood, they delegate the storage and retrieval quantum operations

to the Protocol. The framework as a whole enforces the encrypted cloning protocol's one-time decryption constraint alongside the no-cloning and no-measurement constraints of quantum mechanics.

Validation across ideal and noisy simulations confirms that the framework faithfully reproduces both theoretical and experimental baselines. Under ideal simulations, the retrieval fidelity reaches 1.0, individual qubit purity matches the theoretical 0.5, and total qubit count adheres to the $m(2n + 1)$ scaling across all 800 trials. On the other hand, under noisy simulations, the IBM Heron R2 noise model, which omits cross-talk and other hardware effects, qualitatively reproduces the fidelity degradation trend reported by Yamaguchi et al. (2026) on real quantum hardware. Crucially, timing analysis shows that the classical bookkeeping introduced by QArray and QStack incurs no measurable computational overhead relative to the raw Protocol because all three components produce identical circuits for the same set of identical tasks.

The framework's scope is bounded by the limitations of currently available quantum hardware and simulation tooling. Qiskit Aer simulator imposed a practical ceiling of approximately 30 qubits. Therefore, all trials were designed to use fewer than 30 qubits. In addition, we were able to deploy only a single-core storage model due to limitations of personal computers; distributed quantum simulators were not used to simulate the multi-cloud storage envisioned by Yamaguchi and Kempf (2026). Moreover, we discussed the fundamental issues with currently available NISQ hardware, where coherence times are far shorter than any practical cloud storage retention requirement, and distributed quantum computing is a new research field that has recently gained traction. Thus, our framework should be understood as a software abstraction whose operational deployment depends on hardware capabilities that do not yet exist.

Future work should extend the framework to distributed quantum computers as inter-QPU networking and distributed compilers mature, enabling the multi-cloud application originally proposed by Yamaguchi and Kempf (2026). The map-based register management within the Protocol class was intentionally designed to allow additional data structure abstractions, such as queues or hash maps, to be built without modifying the underlying implementation. The current circuit-cutting utility requires the user to

manually identify barrier indices, but could be replaced with an automated partitioner that detects natural store-retrieve boundaries. In the distant future, when long coherence quantum memory and distributed quantum computers become viable, software frameworks like the one presented here will be essential for translating low-level protocol guarantees into the abstractions that real-world quantum encrypted multi-cloud storage applications will require.

References

AbuGhanem, M. (2025). IBM quantum computers: evolution, performance, and future directions. *The Journal of Supercomputing*, *81*(5). https://doi.org/10.1007/s11227-025-07047-7

Barnum, H., Caves, C. M., Fuchs, C. A., Jozsa, R., & Schumacher, B. (1996). Noncommuting mixed states cannot be broadcast. *Physical Review Letters*, *76*(15), 2818–2821. https://doi.org/10.1103/PhysRevLett.76.2818

Bužek, V., & Hillery, M. (1996). Quantum copying: Beyond the no-cloning theorem. *Physical Review A*, *54*(3), 1844–1852. https://doi.org/10.1103/physreva.54.1844

Caleffi, M., Amoretti, M., Ferrari, D., Illiano, J., Manzalini, A., & Cacciapuoti, A. S. (2024). Distributed quantum computing: A survey. *Computer Networks*, *254*, 110672–110672. https://doi.org/10.1016/j.comnet.2024.110672

Das, A., & DiAdamo, S. (2023). *Diskit documentation*. https://interlin-q.github.io/diskit/index.html

Duan, L., & Guo, G.-C. (1998). Probabilistic Cloning and Identification of Linearly Independent Quantum States. *Physical Review Letters*, *80*(22), 4999–5002. https://doi.org/10.1103/physrevlett.80.4999

Fujiwara, M., Waseda, A., Nojima, R., Moriai, S., Ogata, W., & Sasaki, M. (2016). Unbreakable distributed storage with quantum key distribution network and password-authenticated secret sharing. *Scientific Reports*, *6*(1). https://doi.org/10.1038/srep28988

Google Quantum AI. (2024). *Choosing hardware for your qsim simulation*. https://quantumai.google/qsim/choose_hw

Ma, C.-L., Li, D.-D., Li, Y., Wu, Y., Ding, S.-Y., Wang, J., Li, P.-Y., Zhang, S., Chen, J., Zhang, X., Wang, J.-Y., Li, J., Li, Q., Chen, Z.-T., Zhou, L., Zhao, M.-S., & Zhao, Y. (2023). Quantum-secure fault-tolerant distributed cloud storage system. AIP Advances, 13(11). https://doi.org/10.1063/5.0172384

Miller, M., Günther, J., Witteveen, F., Teynor, M. S., Erakovic, M., Reiher, M., Solomon, G. C., & Christandl, M. (2025). *phase2: Full-State Vector Simulation of Quantum Time Evolution at Scale*. https://doi.org/10.48550/arxiv.2504.17881

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press.

Phalak, K., Chatterjee, A., & Ghosh, S. (2023). Quantum Random Access Memory for Dummies. *Sensors*, *23*(17), 7462. https://doi.org/10.3390/s23177462

SreramanM. (2025). *GitHub - SreramanM/DQCS: Distributed quantum computing simulator*. https://github.com/SreramanM/DQCS/

Werner, R. F. (1998). Optimal cloning of pure states. *Physical Review A*, *58*(3), 1827–1832. https://doi.org/10.1103/physreva.58.1827

Yamaguchi, K., & Kempf, A. (2026). Encrypted Qubits Can Be Cloned. *Physical Review Letters*, *136*(1). https://doi.org/10.1103/y4y1-1ll6

Yamaguchi, K., Rullkötter, L., Shehzad, I., Wagner, S. J., Tutschku, C., & Kempf, A. (2026). *Experimental demonstration that qubits can be cloned at will, if encrypted with a single-use decryption key*. https://doi.org/10.48550/arxiv.2602.10695